

Modelování struktury

Karel Richta

Katedra technických studií
Vysoká škola polytechnická
Jihlava

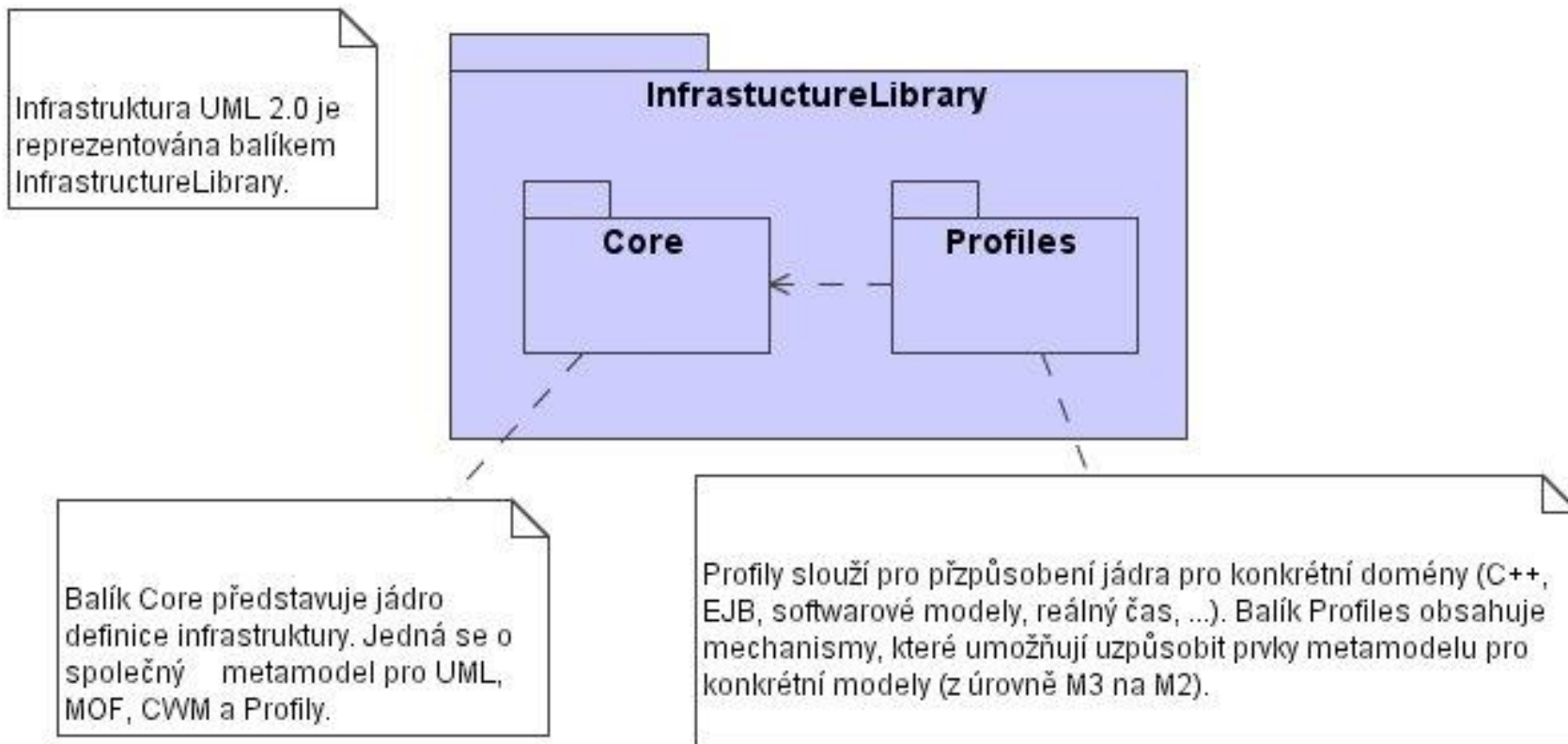
© Karel Richta, 2024

Softwarové inženýrství, SWI
03/2024, Lekce 6

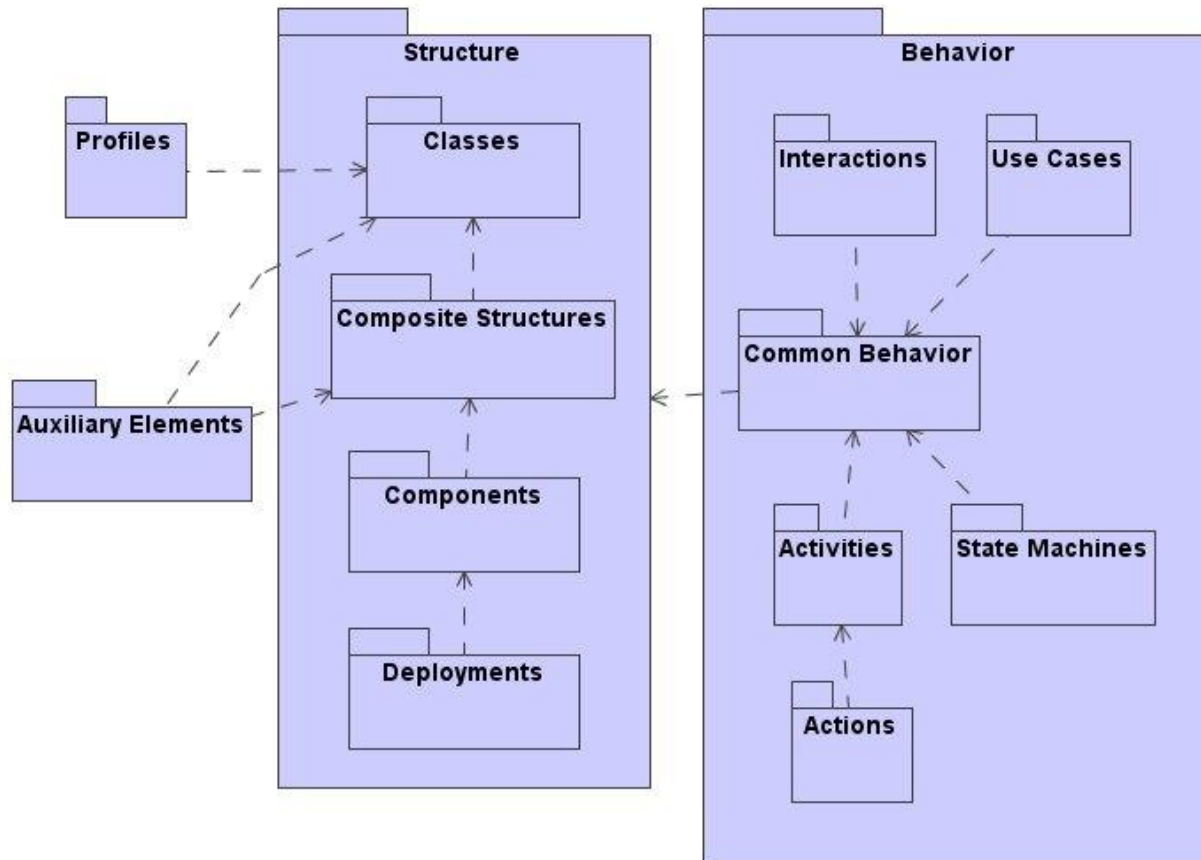
<https://moodle.vspj.cz/course/view.php?id=202893>



Infrastruktura UML



Superstruktura UML



Směr pohledu na systém dle UML

- **Diagramy popisující strukturu** – diagramy tříd, objektů, kompozitní struktury, komponent a nasazení.
- **Diagramy popisující chování** – model jednání, scénáře, diagramy aktivit, komunikace objektů, stavové diagramy.
- **Diagramy pro správu modelů** – balíky (packages), subsystémy, rámce, diagramy komponent.

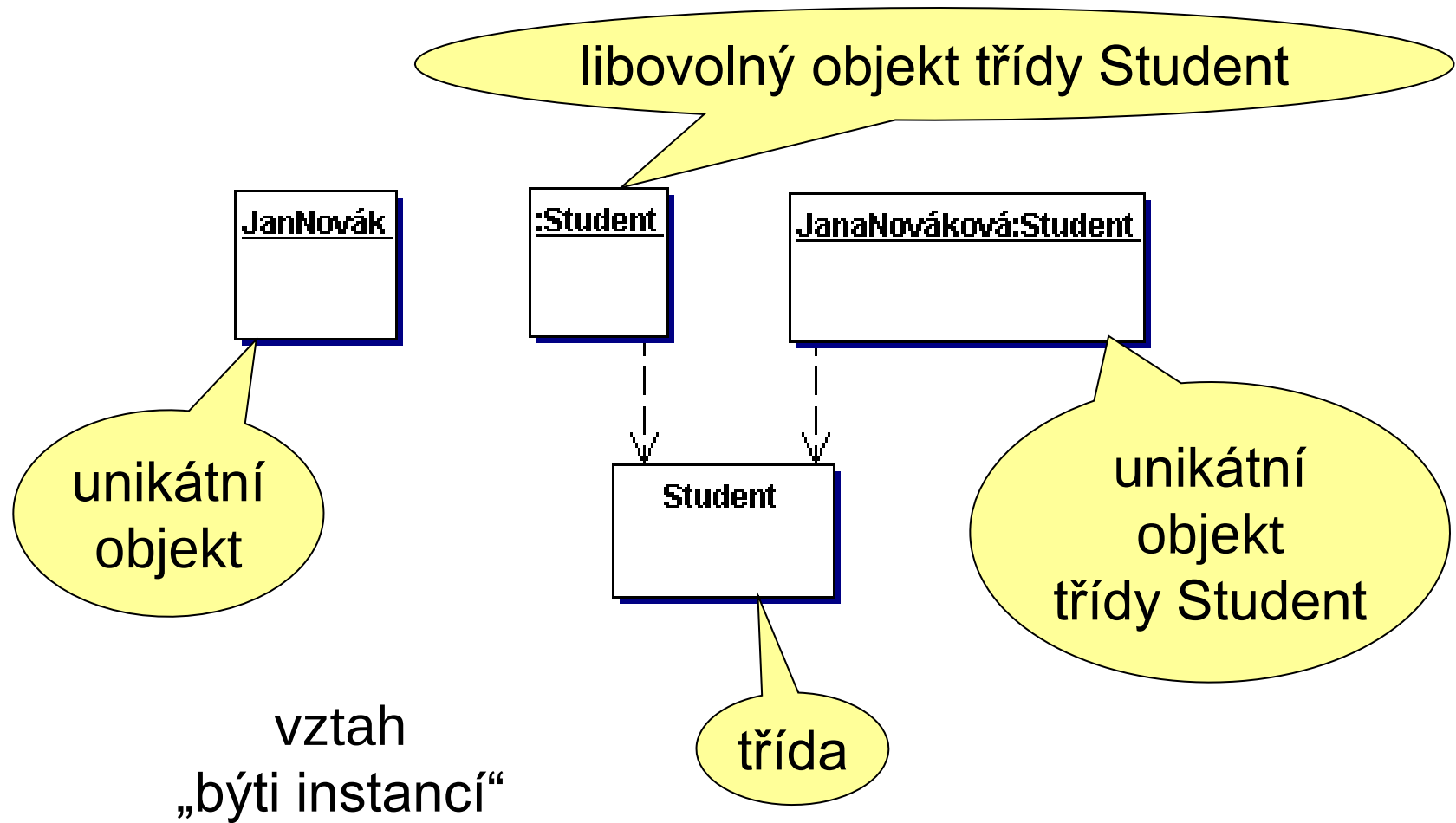


Diagramy objektů

- **Objekt** je pojem, abstrakce, nebo věc s dobře definovanými hranicemi a významem
- Každý objekt má tři charakteristiky: identitu, stav a chování.
 - **Stav** objektu je jedna z možných situací, ve kterých se objekt může nacházet. Stav objektu se může měnit a je definován sadou vlastností - atributů a vztahy.
 - **Chování** určuje, jak objekt reaguje na žádosti jiných objektů a vyjadřuje vše, co může objekt dělat. Chování je implementováno sadou operací (metod).
 - **Identita** znamená, že každý objekt je jedinečný.

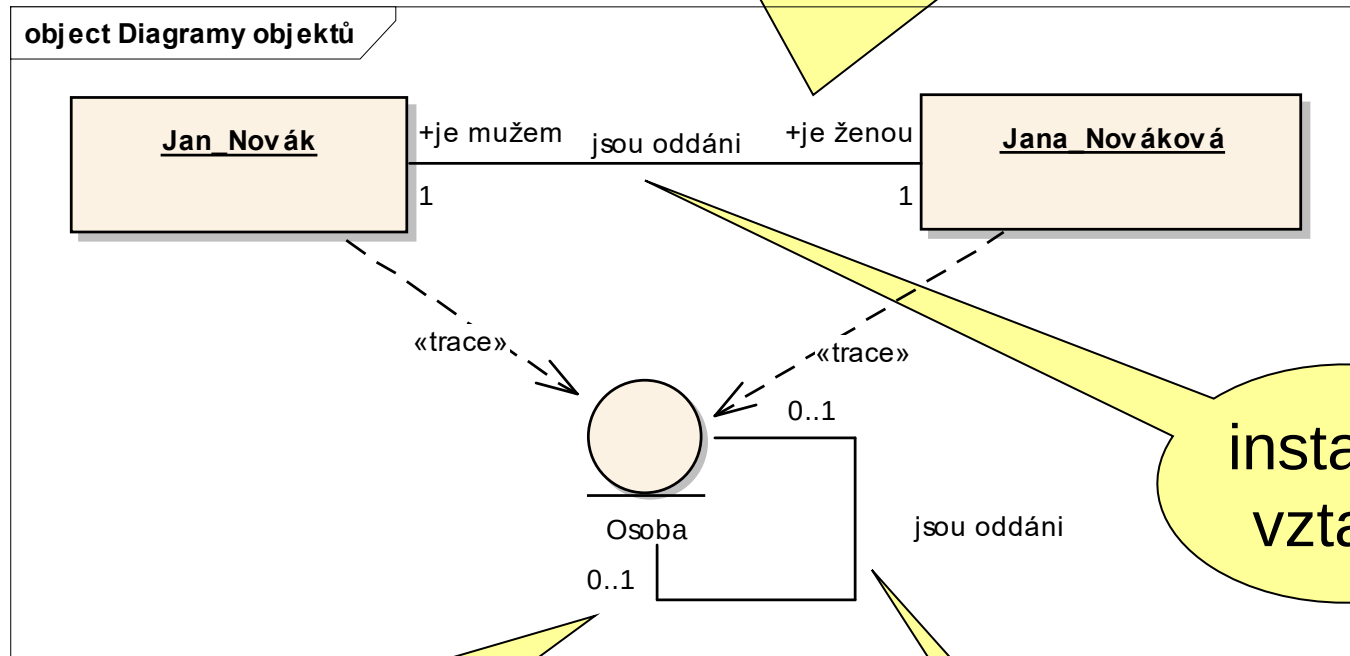


Notace objektů



Vztahy mezi objekty

instance vztahu je vždy 1..1

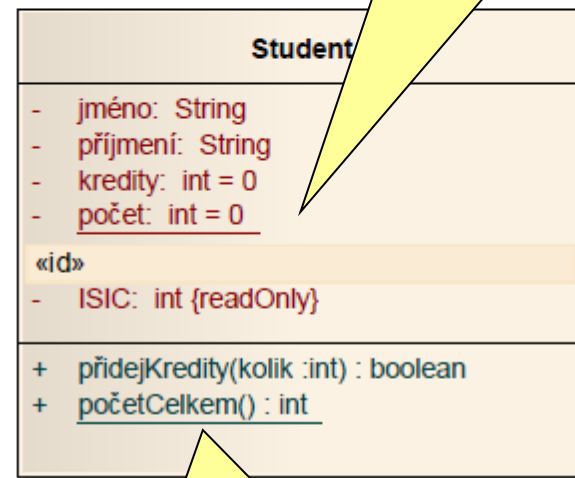
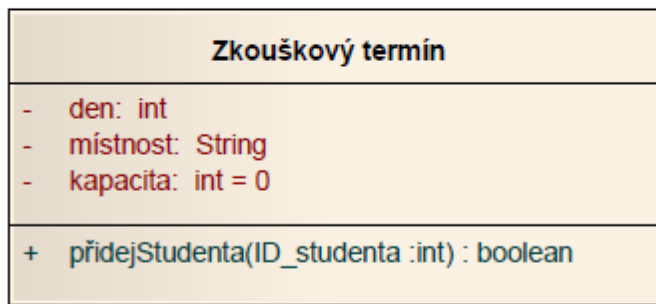
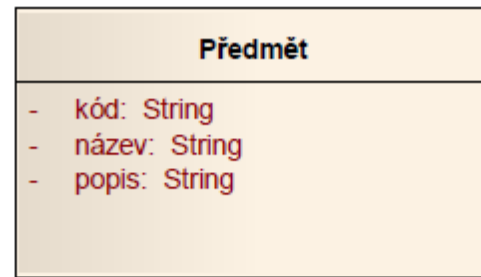
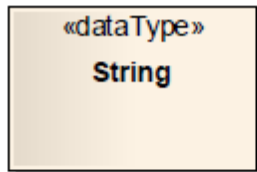


instance vztahu

vztah má kardinalitu

vztah

Notace tříd



statický atribut

statická metoda

Diagramy tříd

Pro definici **atributů** se používá notace:

```
viditelnost jméno [ násobnost ] : typ  
= počáteční hodnota { omezení, vlastnost }
```

Pro definici **operací** (metod) se používá notace:

```
viditelnost jméno(druh jméno: typ, ...) : typ  
{ omezení, vlastnost }
```

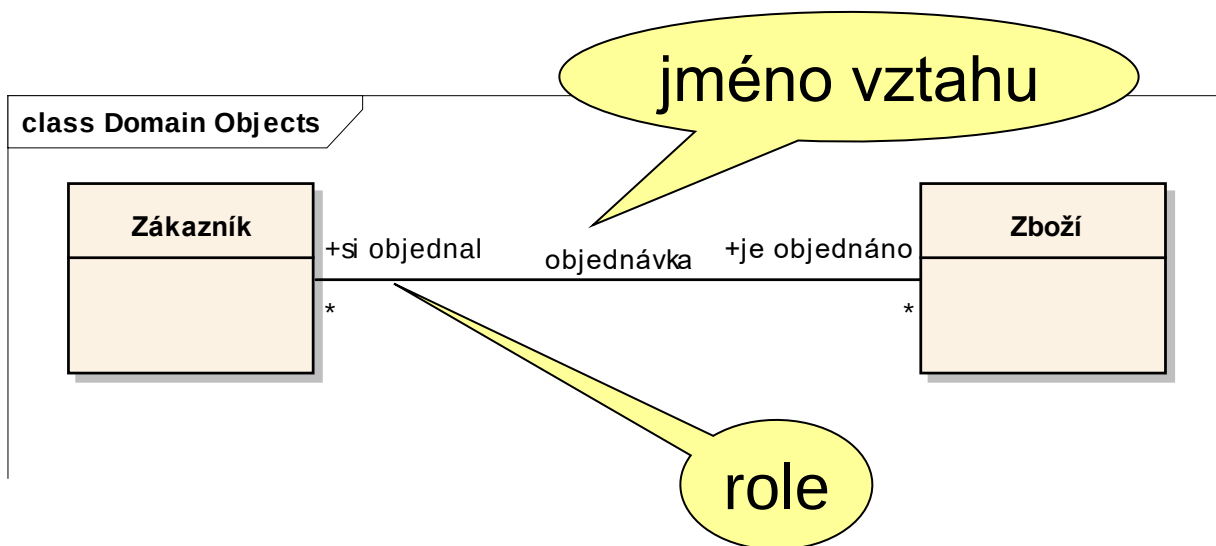
Viditelnost: + ... public # ... protected - ... private

Násobnost: [a..b] ([0..1] může být NULL)

Druh: in, out, inout (implicitně in)

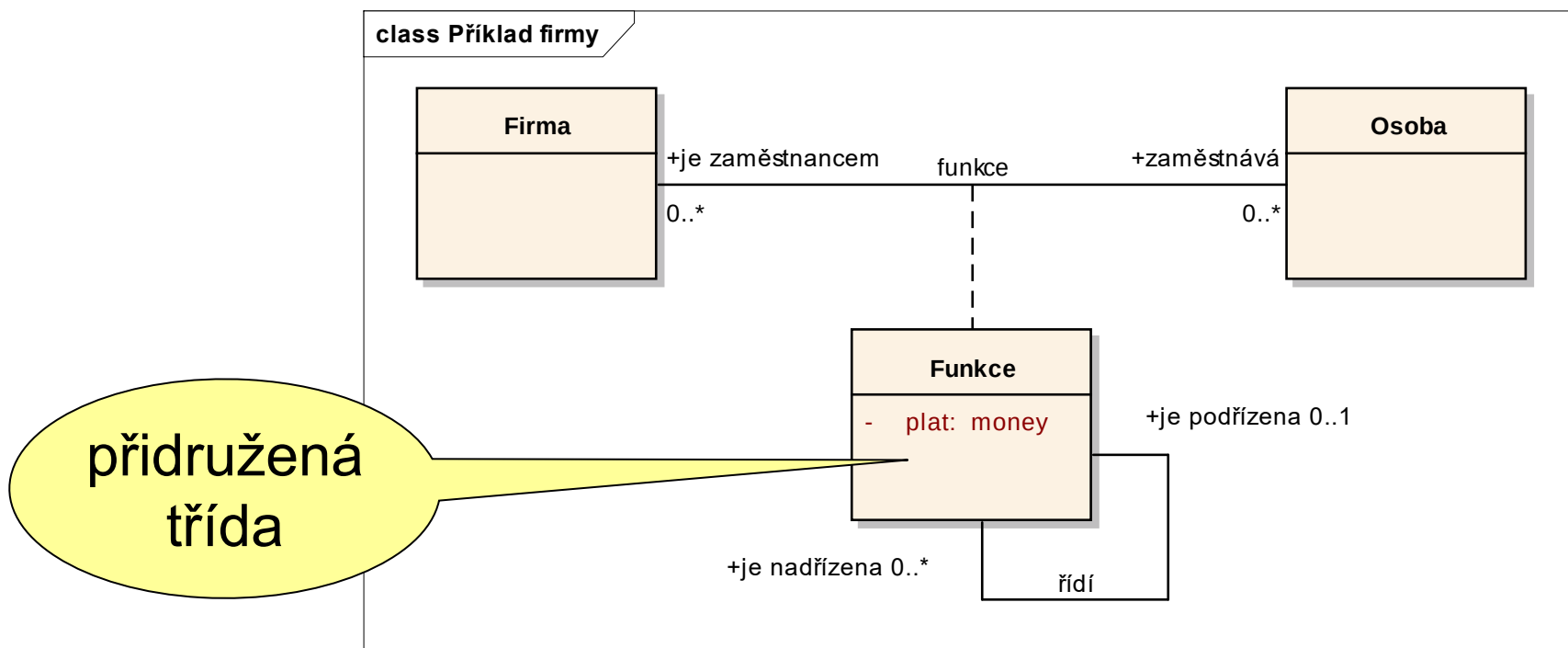
Vztahy mezi třídami

- **vztah (asociace)** - vyznačení možného vztahu mezi objekty
- **role** – označení konce vztahu, které říká, jakou roli objekt ve vztahu hraje
- Pro vyjádření kardinality a volitelnosti se používá notace: **N..M**, kde N a M může být číslo nebo * (samotná * znamená 0..*)



Přidružené třídy (atributy vztahů)

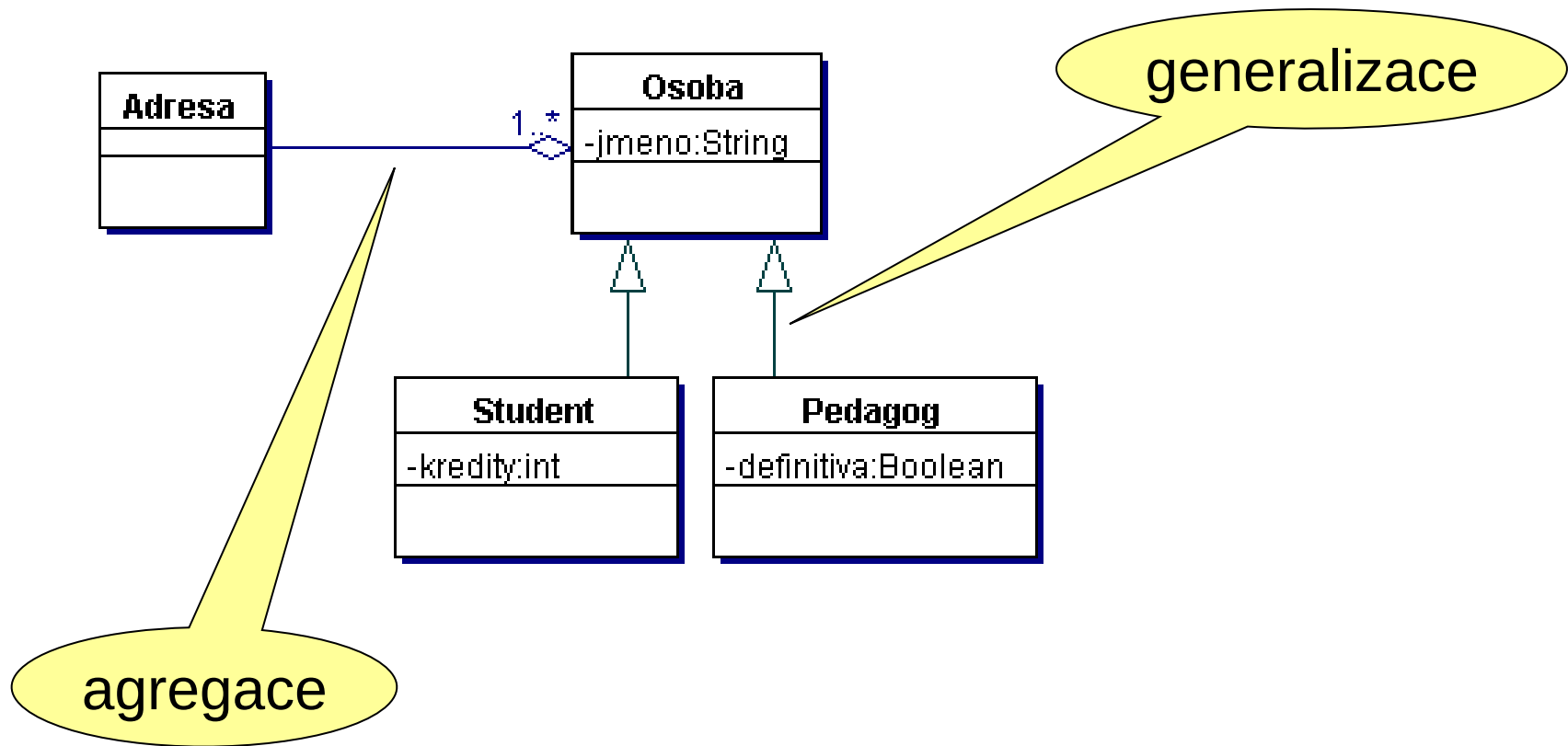
Vztah nemá atributy, ale může být popsán přidruženou třídou



Agregace a generalizace

- **agregace** (*aggregation*) – druh vztahu, kdy jedna třída je součástí jiné třídy - vztah typu celek/část
- **kompozice** (*composition*) – silnější druh agregace - u kompozice je část přímo závislá na svém celku, zaniká se smazáním celku a nemůže být součástí více než jednoho celku
- **generalizace** (*generalization*) – druh vztahu, kdy jedna třída je zobecněním vlastností jiné třídy (jiných tříd) - vztah typu nadtyp/podtyp, generalizace/specializace

Příklad agregace a generalizace

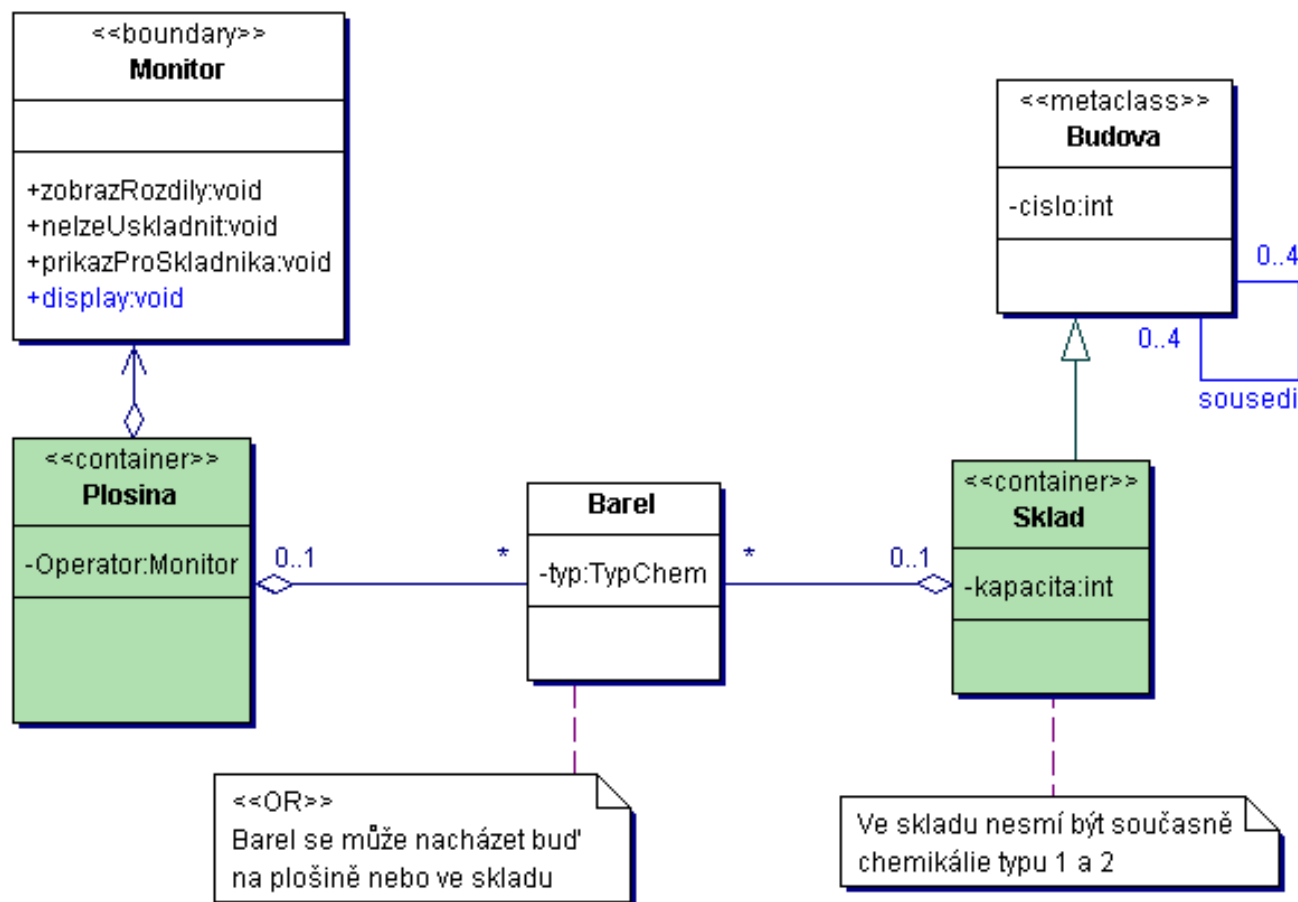


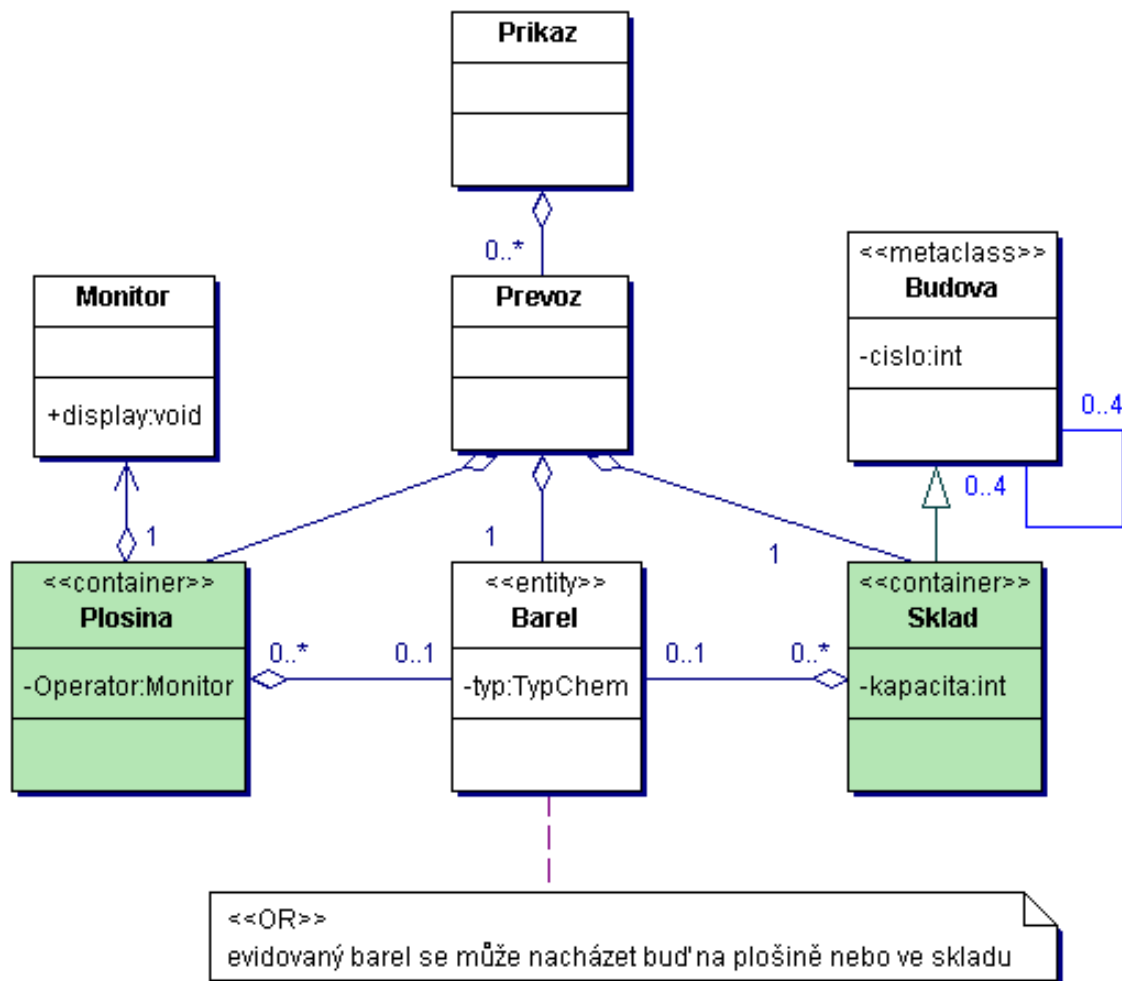
Stereotypy a třídy

Stereotypem lze zavést nový druh prvku modelování. Zde můžeme tvořit nové druhy (skupiny) tříd. Nejčastějšími skupinami (stereotypy) tříd jsou:

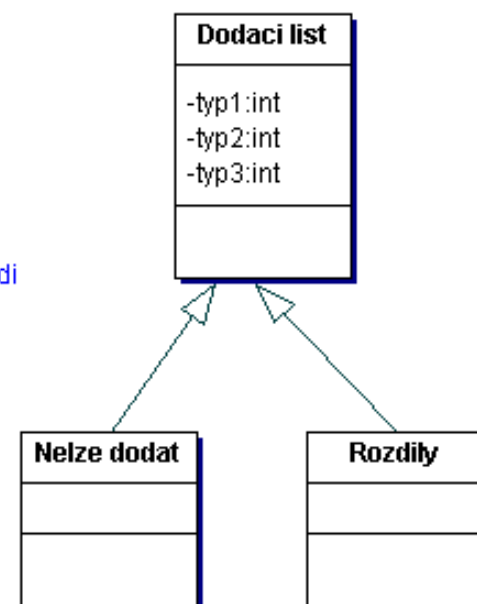
- **entitní třídy** (stereotyp je <<entity>>),
- **třídy rozhraní** (stereotyp je <<boundary>>) a
- **třídy řídicí** (stereotyp je <<control>>).

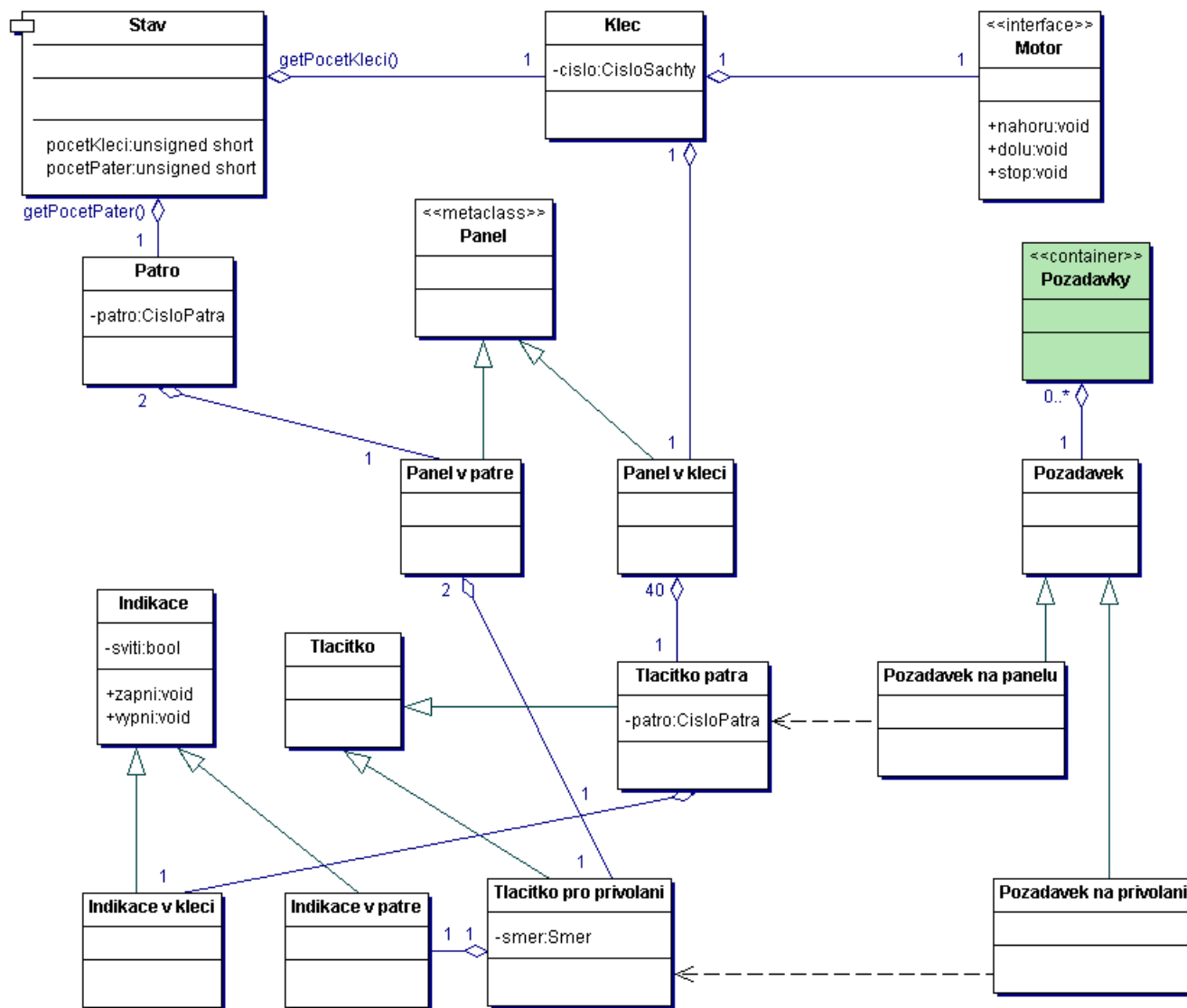
Třídy se stereotypy





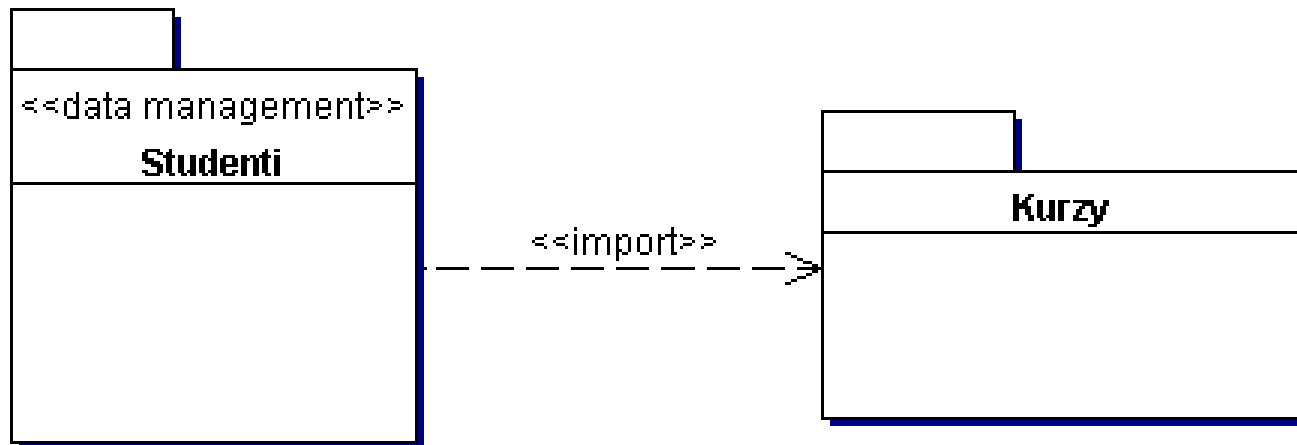
sousedí





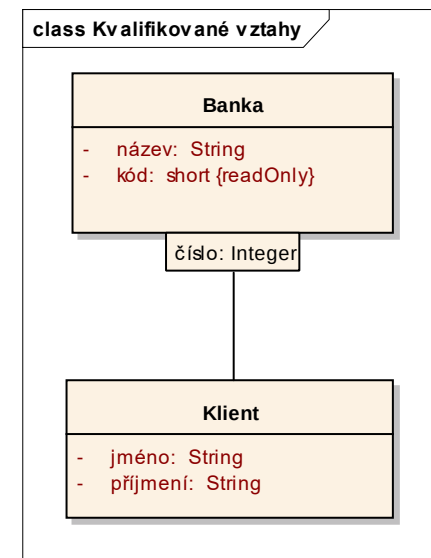
Balíky (packages)

Logická skupina elementů



Další možnosti

- Kvalifikované vztahy
- Navigované vztahy
- Odvozené atributy a vztahy (předznačí se '/')
- „Interface“ - třída bez atributů



Generiky a generické instance

- Parametrizované třídy (generiky)
- Generické instance

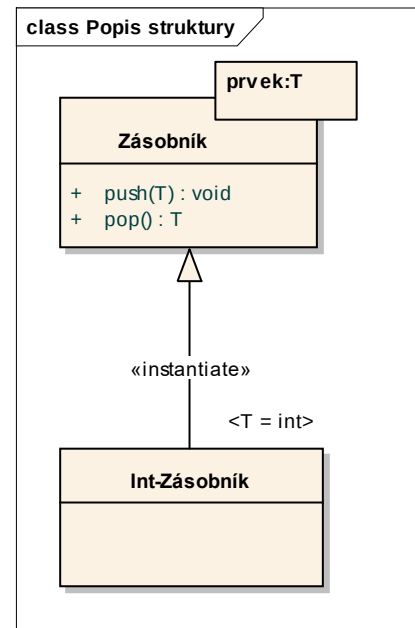


Diagram tříd – „property“

- Nový prvek metamodelu diagramu tříd je vlastnost (property). To může být atribut nebo konec jednosměrného vztahu

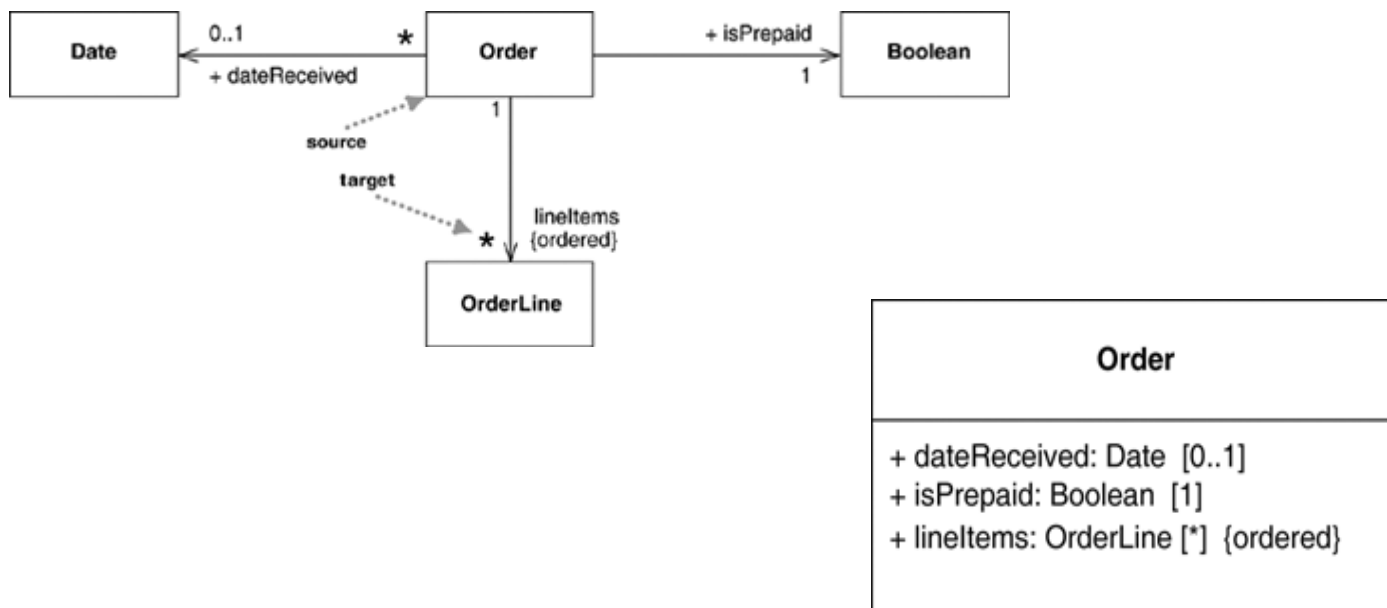


Diagram tříd – „property strings“

- Umožňuje přidat k vlastnosti (property) její další specifikaci. Jako každé omezení se zapisuje ve složených závorkách.



Diagram tříd – „property strings“

- Specializace vztahu {redefines}

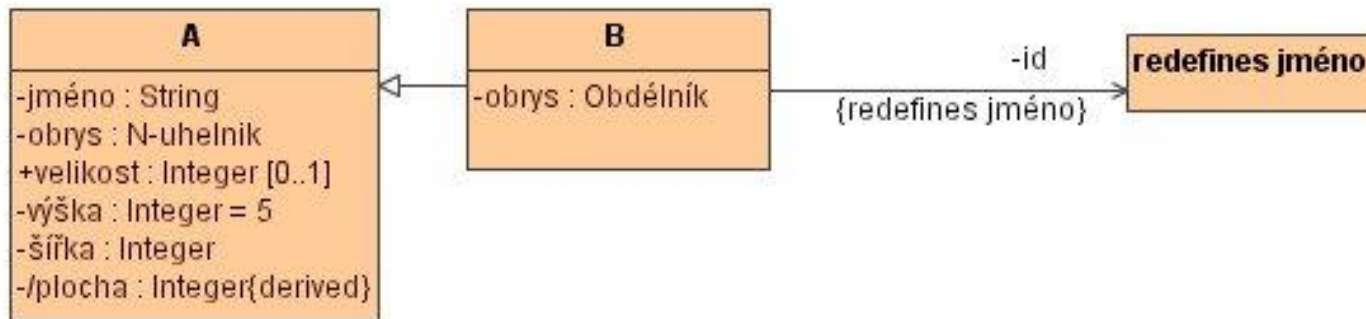
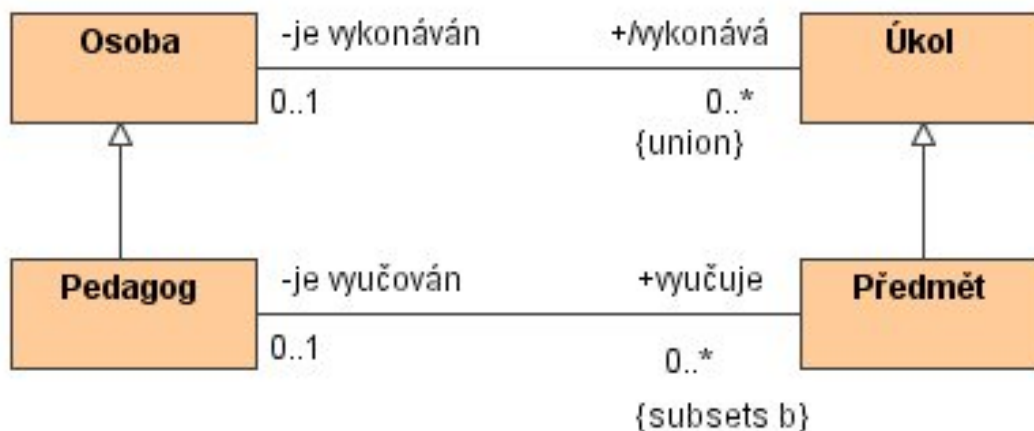
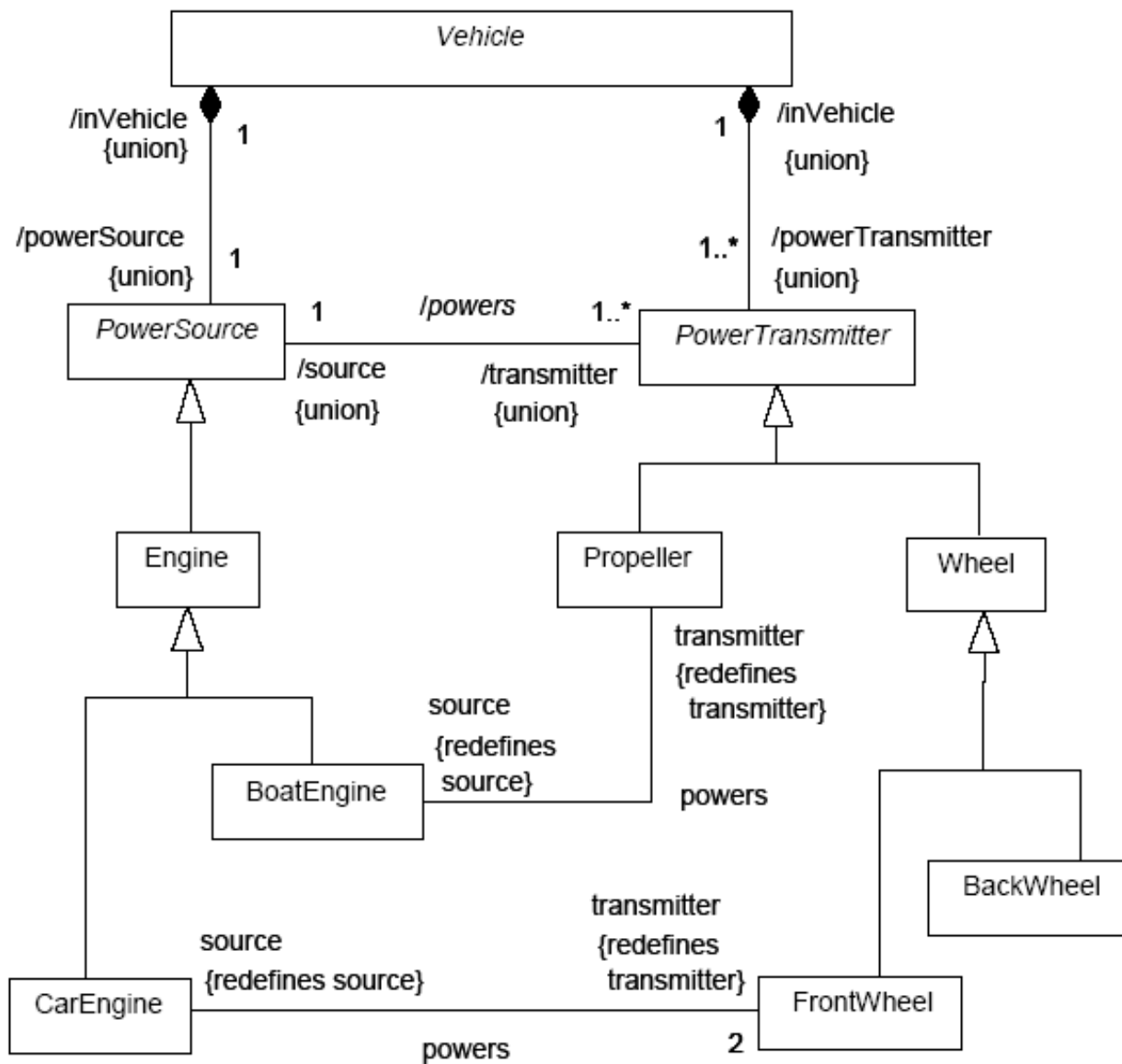


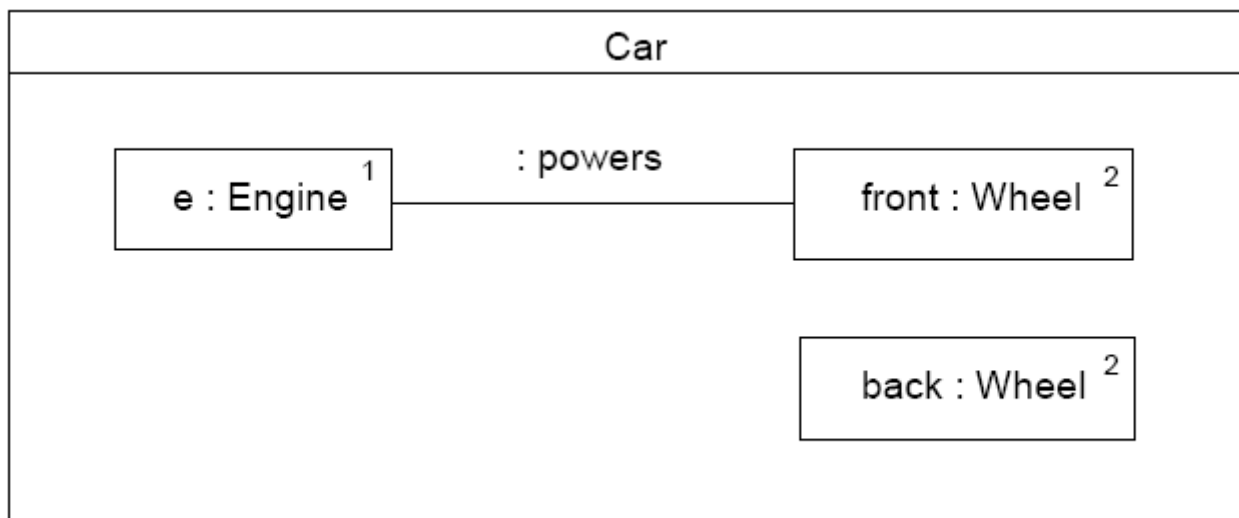
Diagram tříd – „property strings“

- Omezení specializace konce vztahu {subsets <vlastnost>}, {union}, {bag}, {sequence}, {ordered}, ...



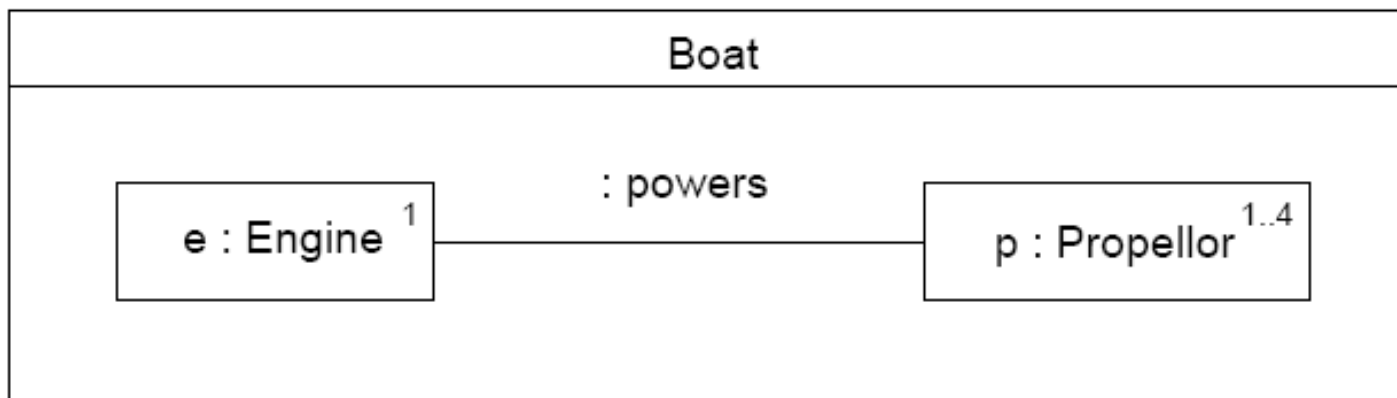


Diagramy kompozitní struktury



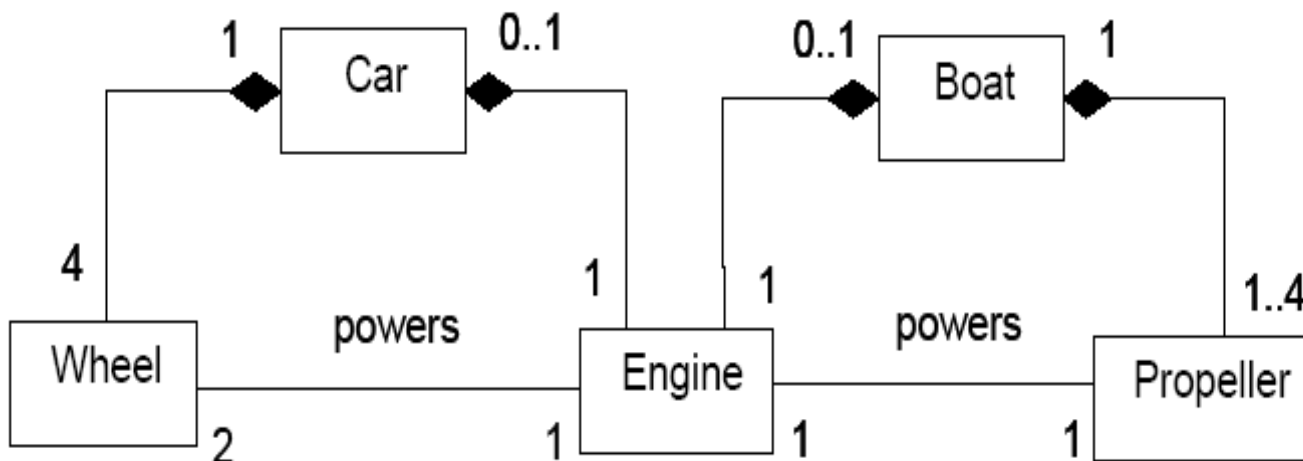
- V každém autě je jeden motor, dvě přední a dvě zadní kola
- Motor pohání přední kola v tomtéž autě
- Motor nemůže pohánět nic jiného

Diagramy kompozitní struktury



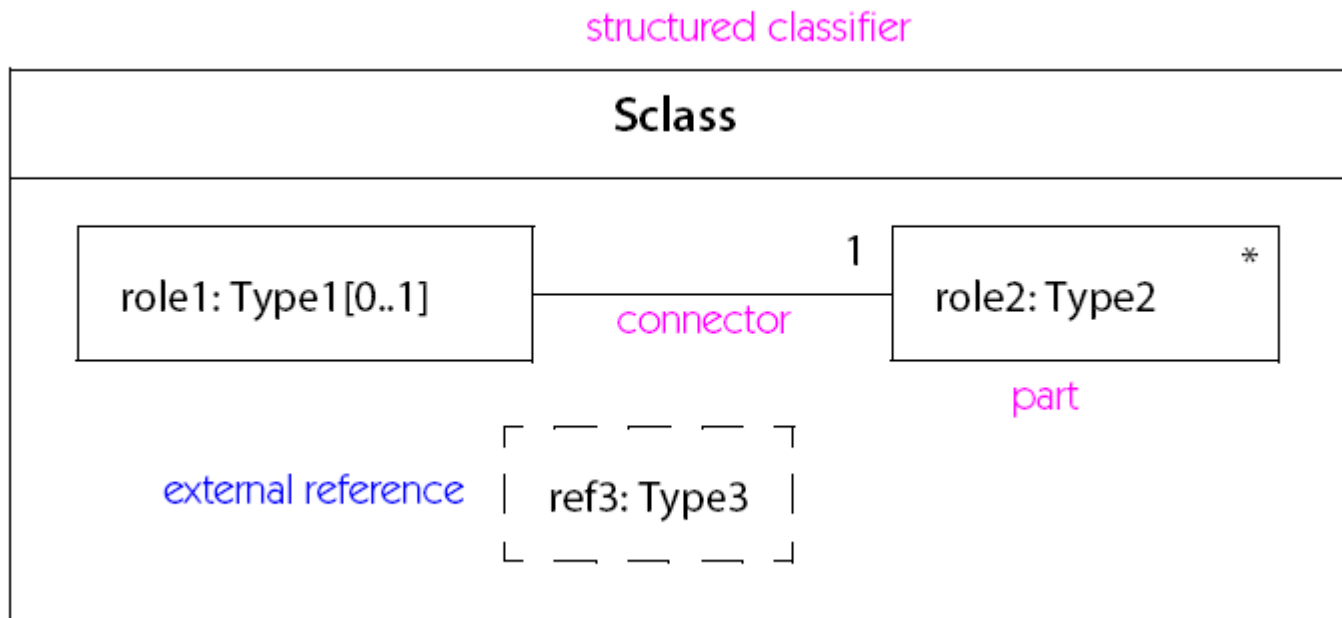
- V každém člunu je jeden motor a několik lodních šroubů
- Motor pohání lodní šrouby v tomtéž člunu
- Motor nemůže pohánět nic jiného

Totéž pomocí tříd ?



- Motor v jednom autě může pohánět kola v jiném autě
- Každý motor pohání kola i lodní šroub
- Motor může pohánět libovolná dvě kola

Notace diagramů kompozitní struktury



Část (Role, Part)

- Instance určitého typu, tvořící část složené struktury.
- Popisuje roli (úlohu, part), kterou instance hraje uvnitř (v rámci) klasifikátoru.
- Část slouží též jako konec vztahu v kontextu složené struktury.
- Část má jméno, typ a multiplicitu.
- Příklad: Přední kolo - front: Wheel[2]

front : Wheel ²

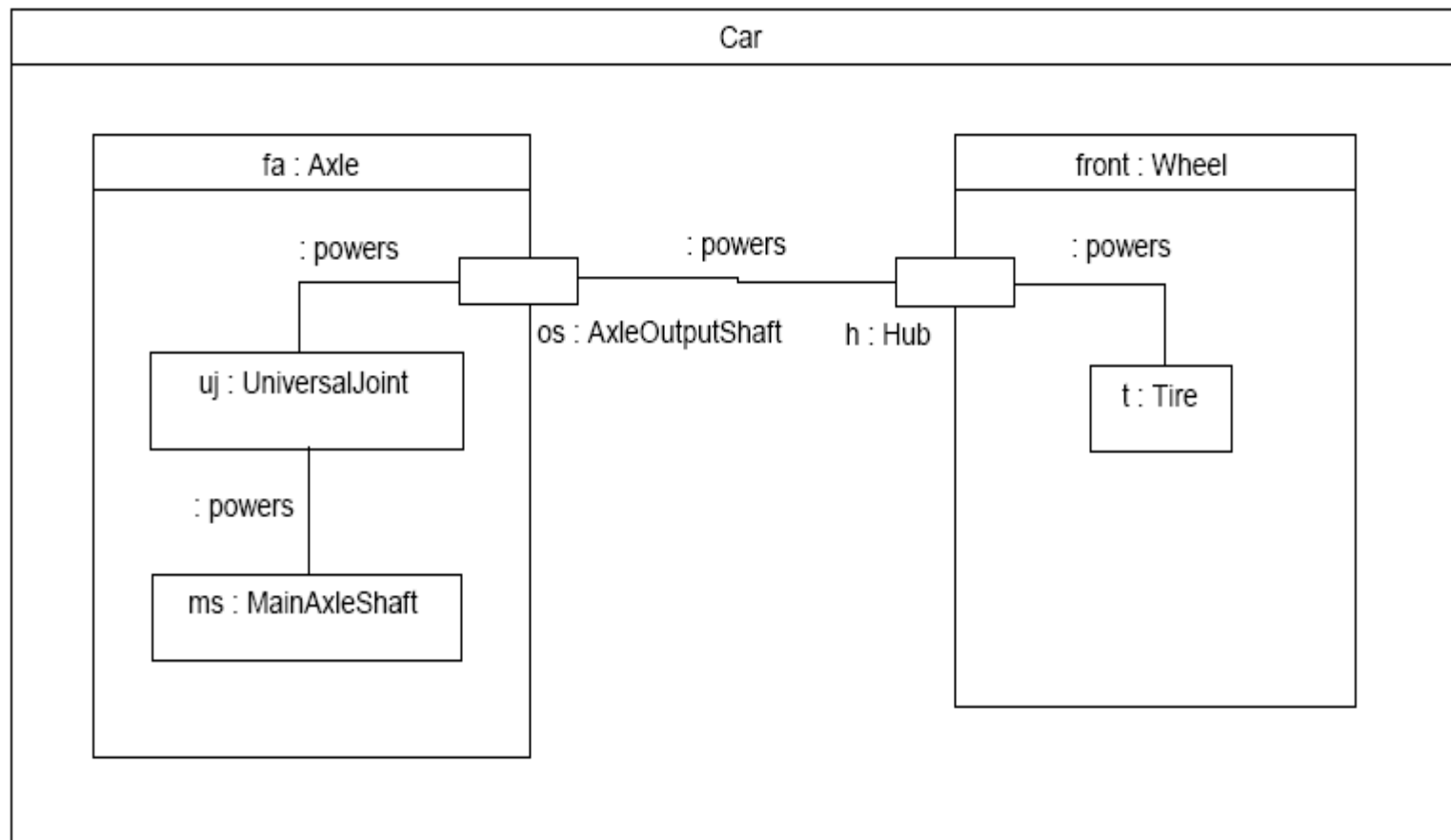
Konektor (Connector)

- Specifikuje instanci vztahu (asociace, vazby, „linku“) v kontextu strukturovaného klasifikátoru.
- Vztah mezi instancemi vystupujícími v rolích v daném strukturovaném klasifikátoru.
- Rozdíl
 - asociace je vztah mezi klasifikátory
 - konektor je vztah mezi rolemi (party)
- Příklad: motor pohání přední kola (:powers)

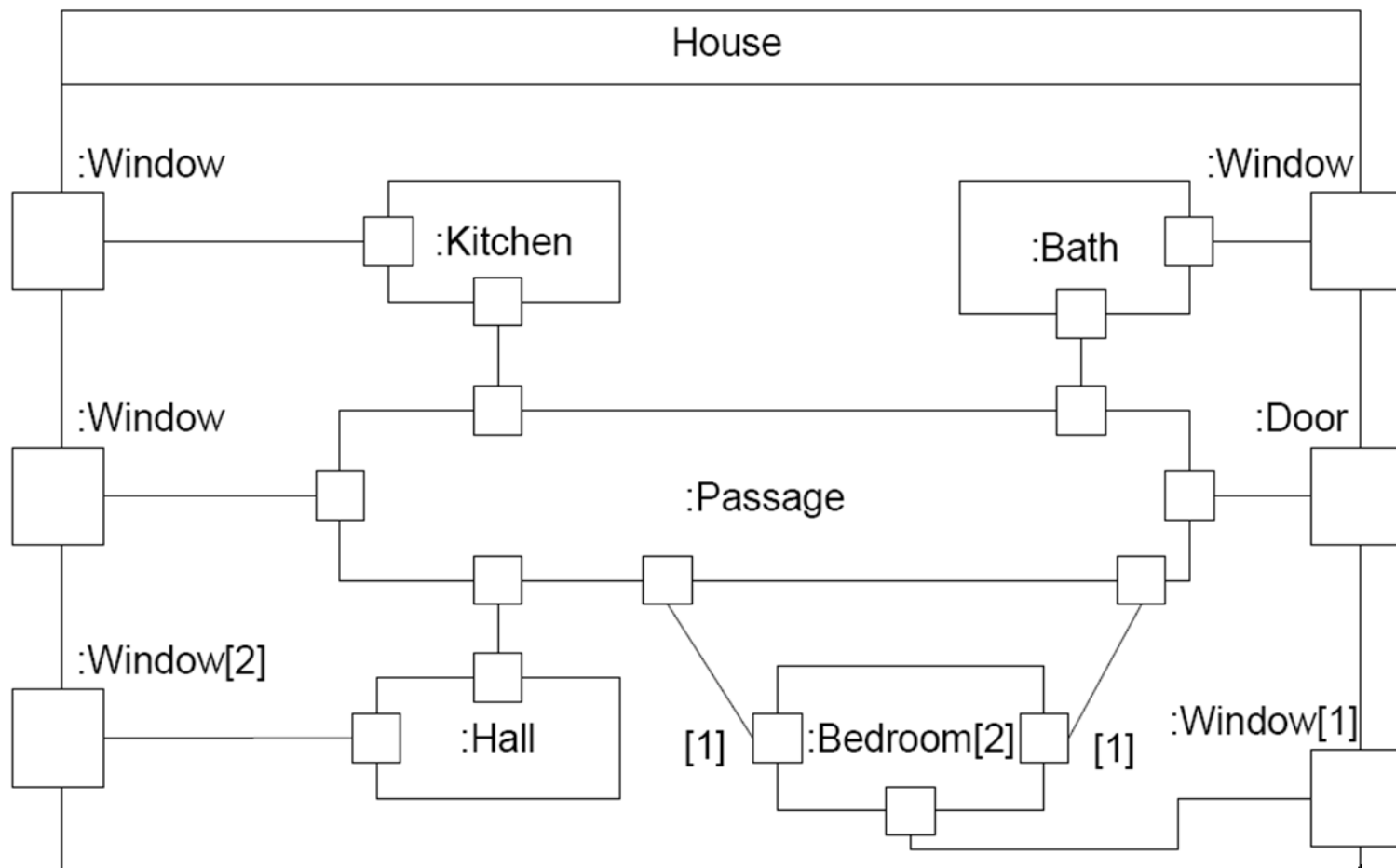
Port

- Role (části), které jsou dostupné zvnějšku.
- Bod interakce mezi klasifikátorem a jeho okolím.
- Zprávy (volání, signály) mohou být směrovány na instance portů (ne na objekt).
- Porty mohou být spojeny s vnitřními prvky (rolemi) nebo se specifikací chování celého objektu.
- Implementace portu zahrnuje rozpoznání události a její předání odpovídajícímu prvku.
- Porty jsou vytvořeny při vytváření instance a zrušeny při jejím zániku.

Příklad použití portu



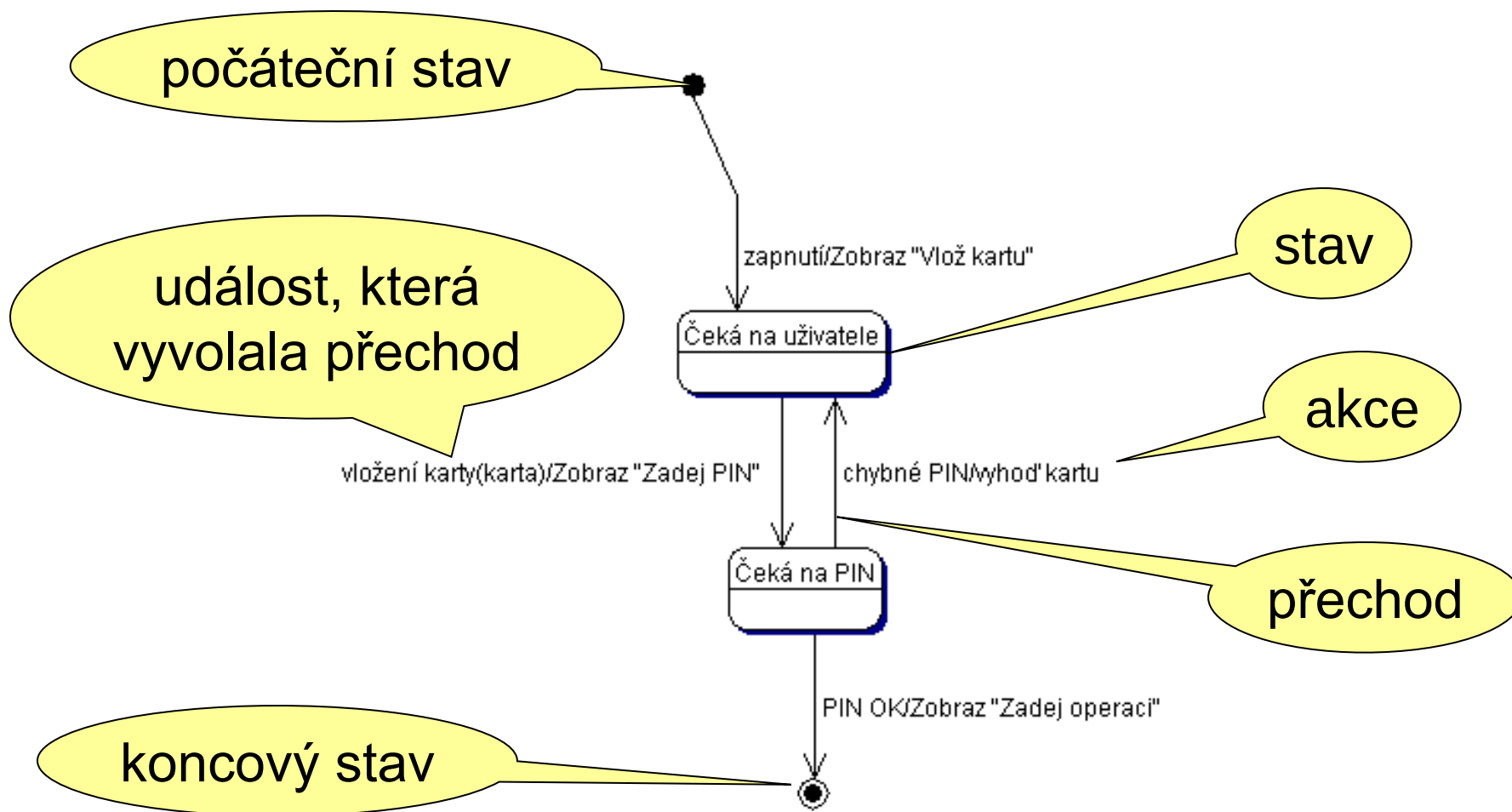
Kompozitní struktura bytu



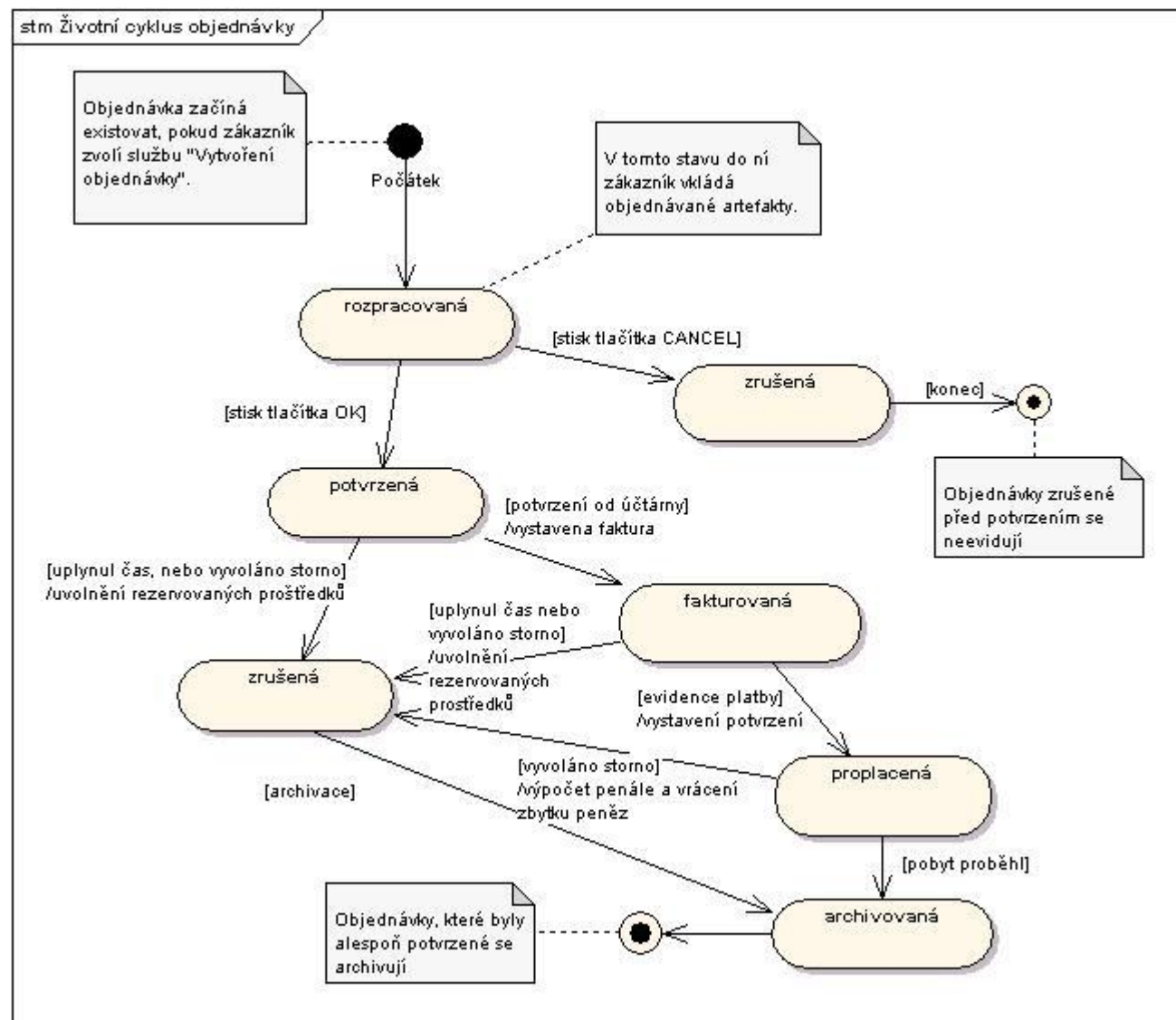
Stavové diagramy

- Slouží k popisu dynamiky systému
- Stavový diagram definuje možné **stavy**, možné **přechody** mezi stavy, **události**, které přechody iniciují, **podmínky** přechodů a **akce**, které s přechody souvisí
- Stavový diagram lze použít pro popis dynamiky objektu (pokud má rozpoznatelné stavy), pro popis metody (pokud známe algoritmus), či pro popis protokolu (včetně protokolu o styku uživatele se systémem)

Notace stavových diagramů



Příklad: Stavový model objed- návky



Stavový stroj

Zahrnuje:

- (konečnou) sadu stavů a regionů – stav může zahrnovat několik regionů (1..N),
- haldu událostí, které nastaly v okolí,
- informaci o aktivních stavech a regionech,
- informaci o vnitřních aktivitách stavů (tzv. „do“ aktivity),
- globální stav stroje (stav proměnných).

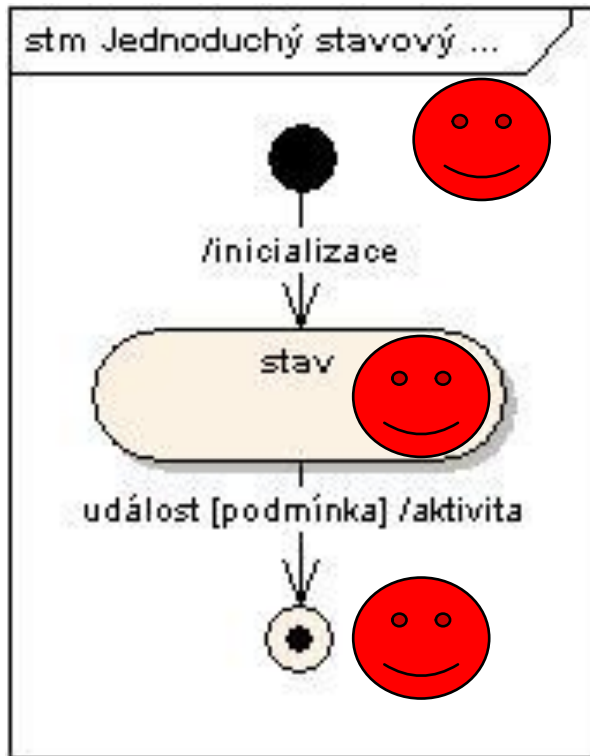
Jak se stavový stroj chová?

- Rozhodne se, zda bude obsluhovat událost nebo vnitřní aktivitu.
- Pokud vybere vnitřní aktivitu, vybere aktivní stav, který má nedokončenou vnitřní aktivitu „do“ a provede jeden krok této aktivity.
- Pokud vybere událost, vybere všechny stavy, které jsou aktivní a které mají výstupní přechod ohodnocený touto událostí.
- Z nich vybere stavy, jejichž výstupní přechod má splnění podmínku.
- Jeden z přechodů uskuteční – deaktivuje vstupní stav, aktivuje výstupní stavy.
- Pokračuje následujícím cyklem, ale až po dokončení předchozího.

Karel Richta (VŠ)

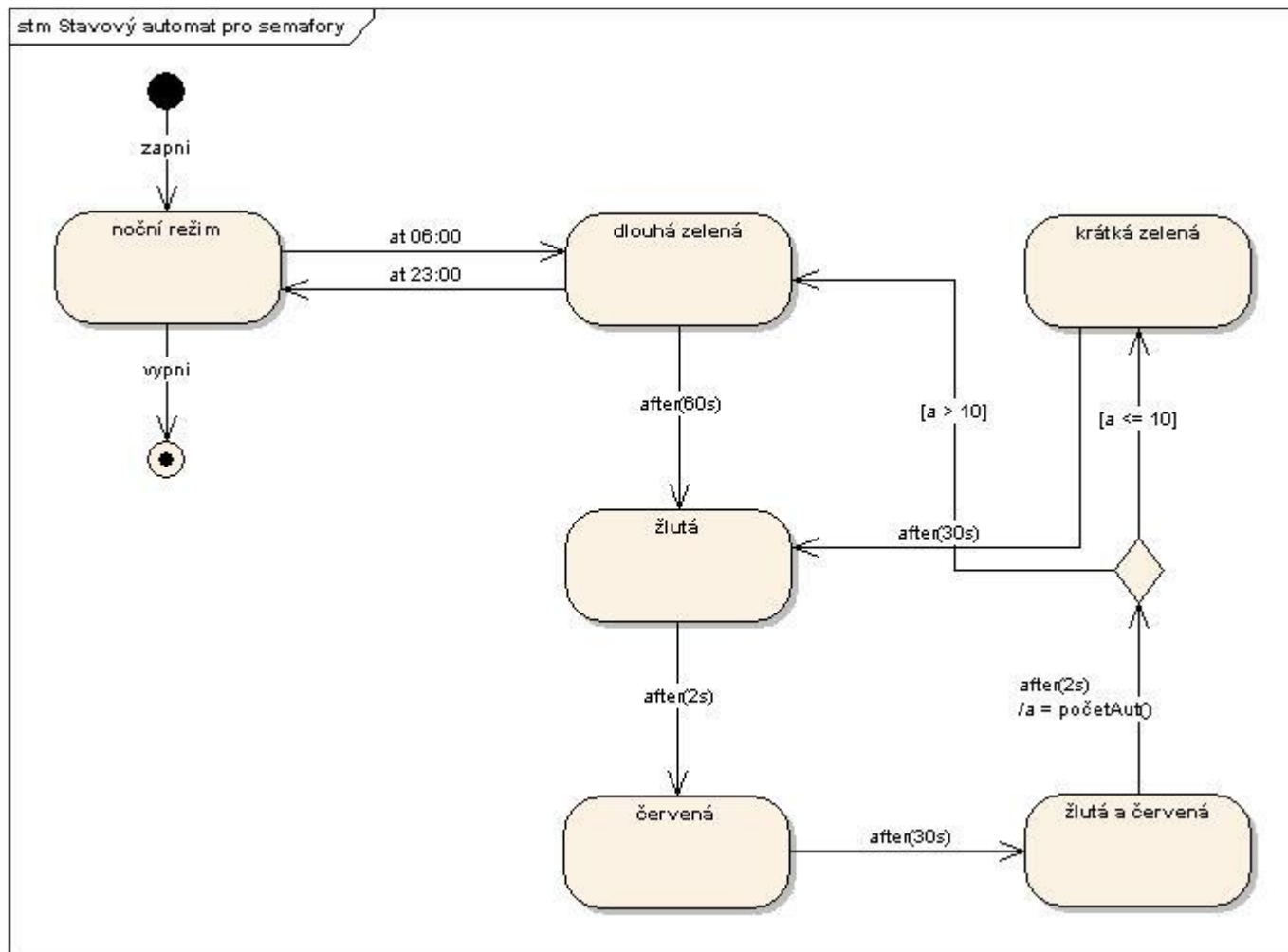


Jednoduchý případ

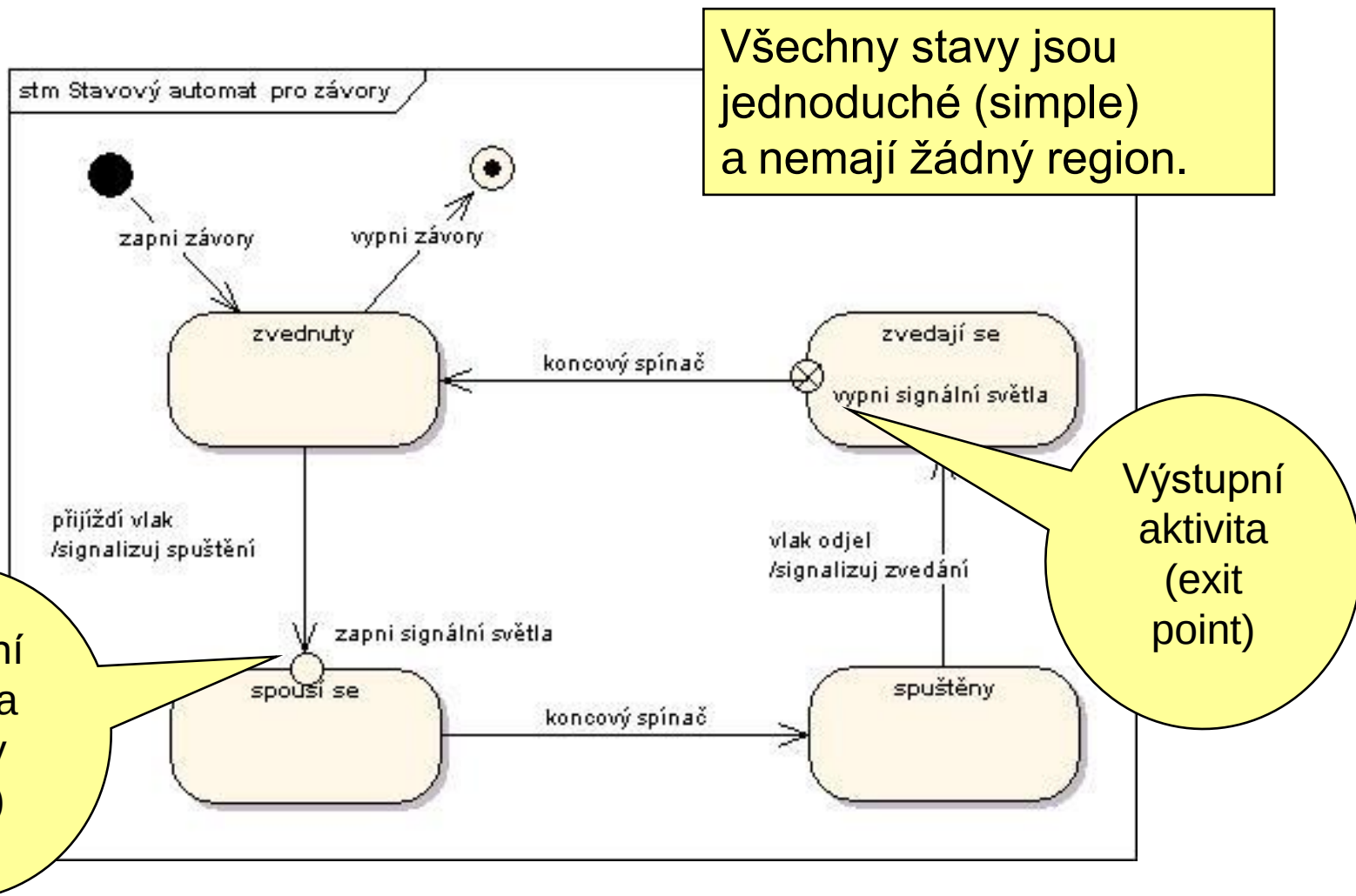


1. Žádný stav není aktivní.
2. Nastala událost spouštějící činnost stroje – na haldě událostí je jen jedna událost.
3. Protože není žádný aktivní stav, vybere se počáteční stav a aktivuje se.
4. Protože je aktivní počáteční stav a nastala událost, deaktivuje se počáteční stav, provede se přechod a inicializace s ním spojená, aktivuje se „stav“.
5. Jakmile nastane „událost“ a je splněna „podmínka“, provede se „aktivita“, deaktivuje se „stav“ a aktivuje se koncový stav – činnost stroje se ukončí.

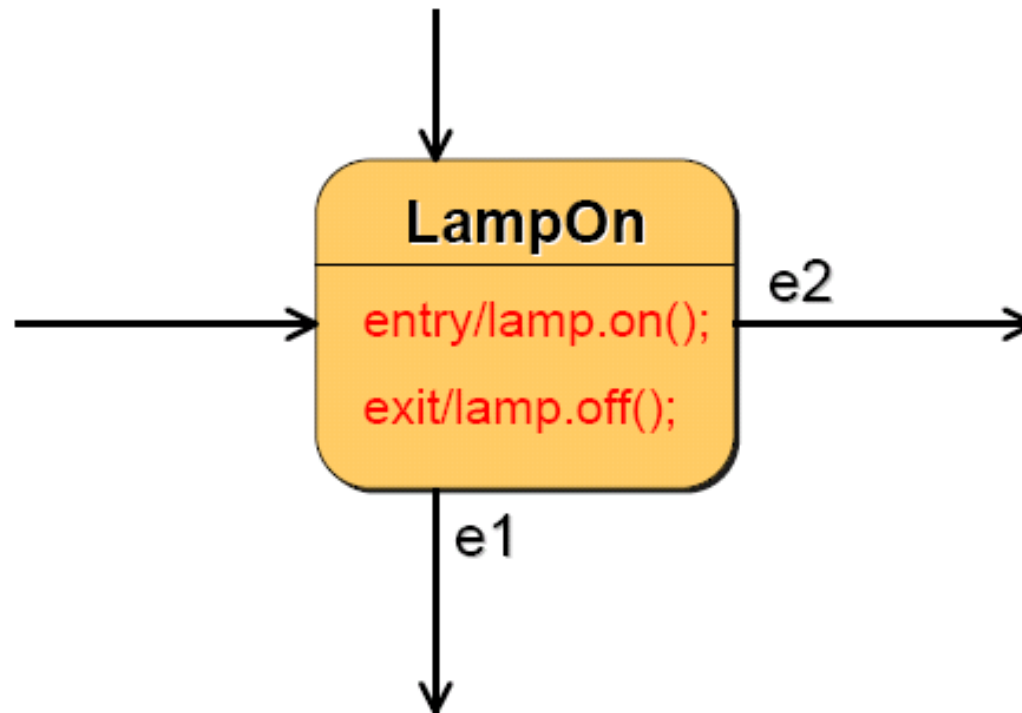
Časové události (at, after)



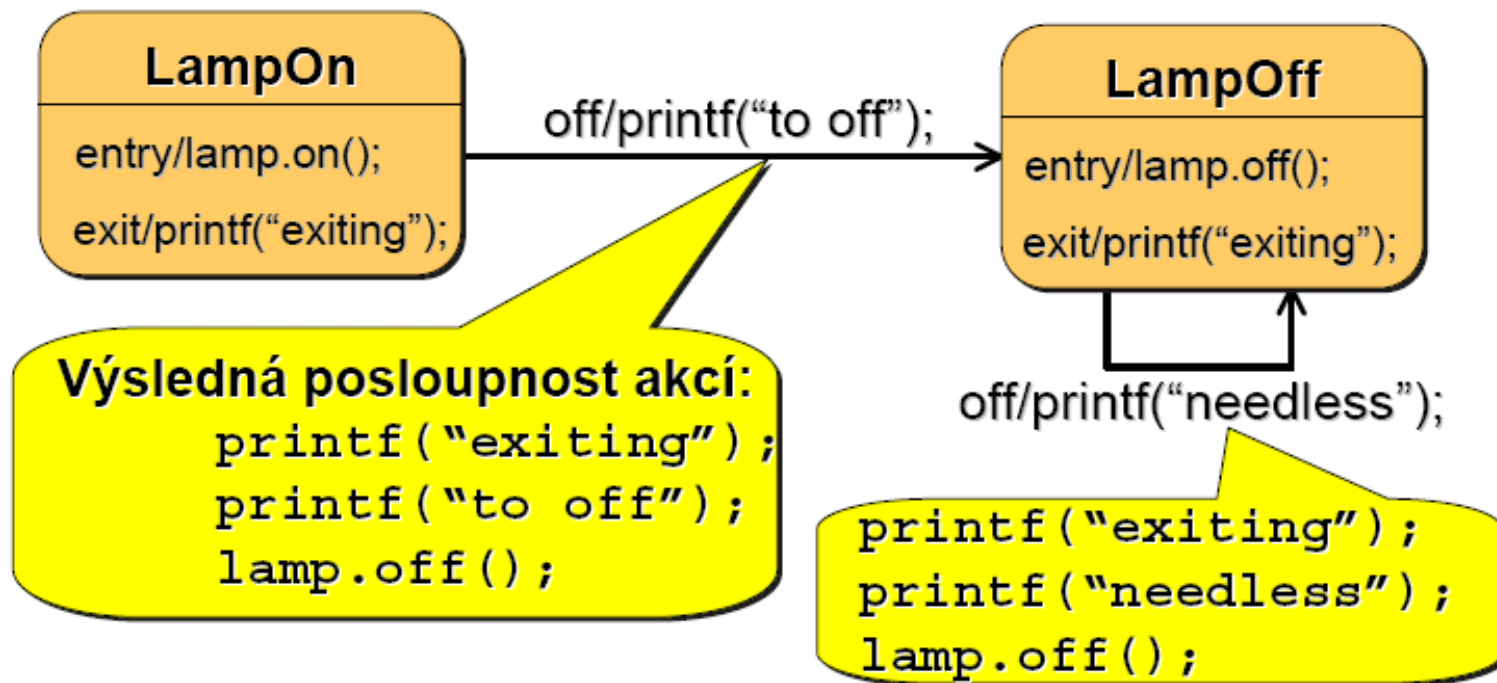
Vstupní a výstupní aktivity: Příklad závor



Vstupní a výstupní aktivity



Pořadí provádění akcí



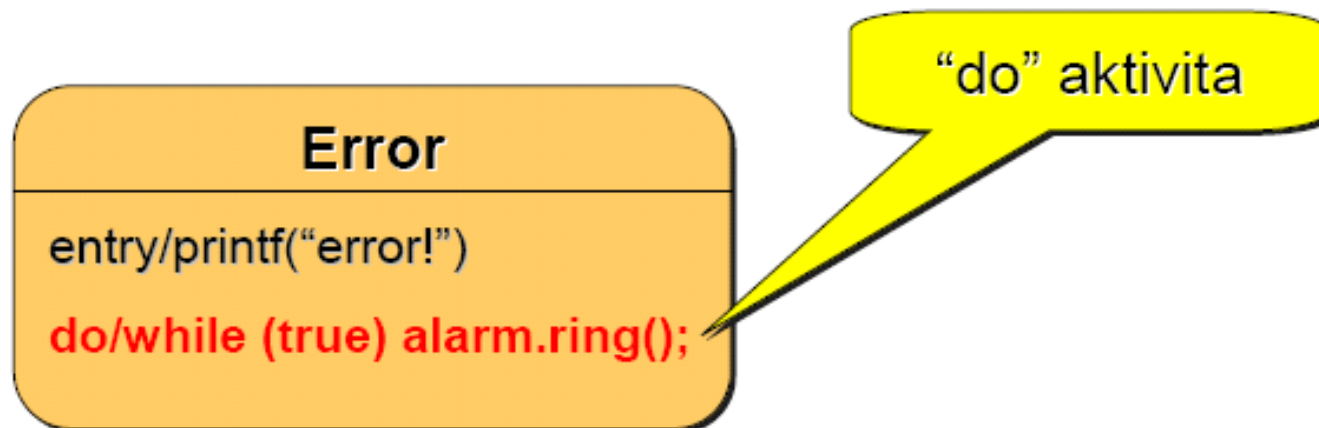
Vnitřní přechody

Vnitřní přechod
spouštěný signálem
“off”

LampOff
entry/lamp.off();
exit/printf(“exiting”);
off/null;

Nedochází k přechodu
do jiného stavu

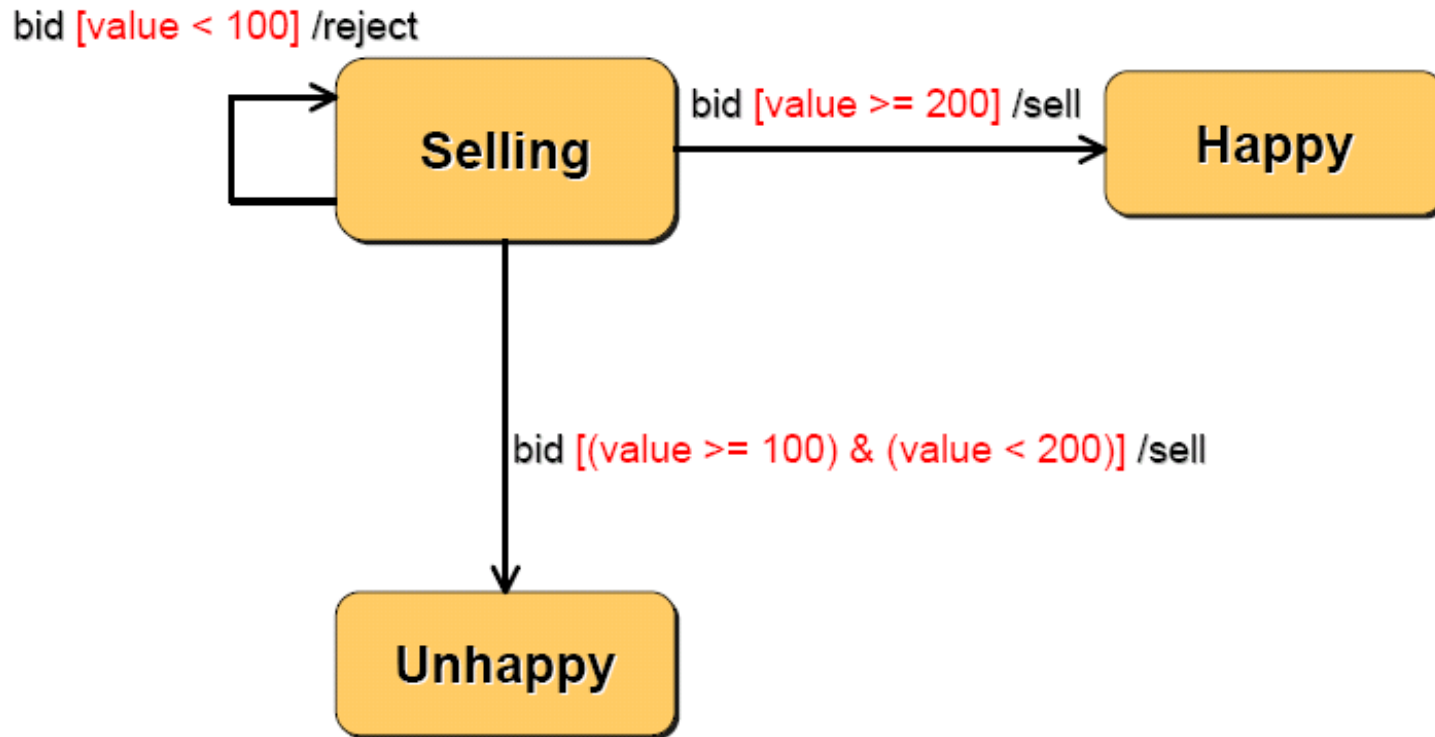
Vnitřní aktivity („do“ aktivity)



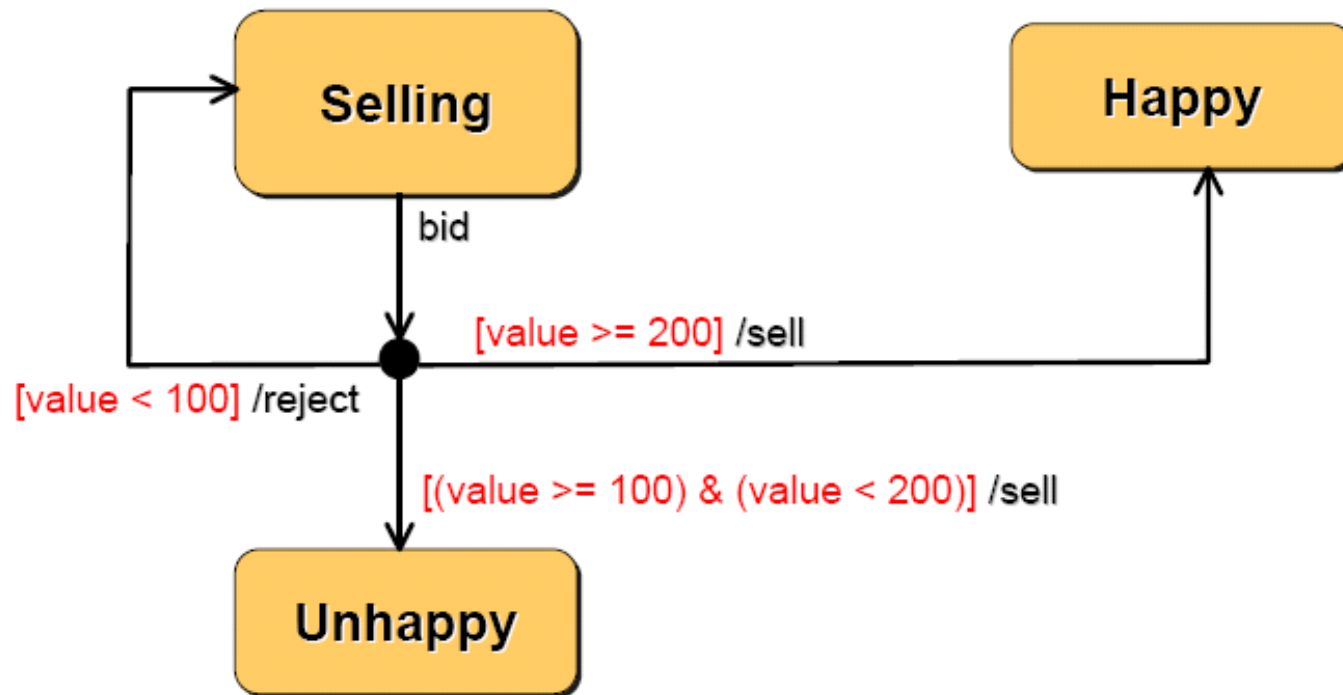
spouští paralelní vlákno prováděné dokud:

- akce neskončí
- nedojde k přechodu do jiného stavu (signálem)

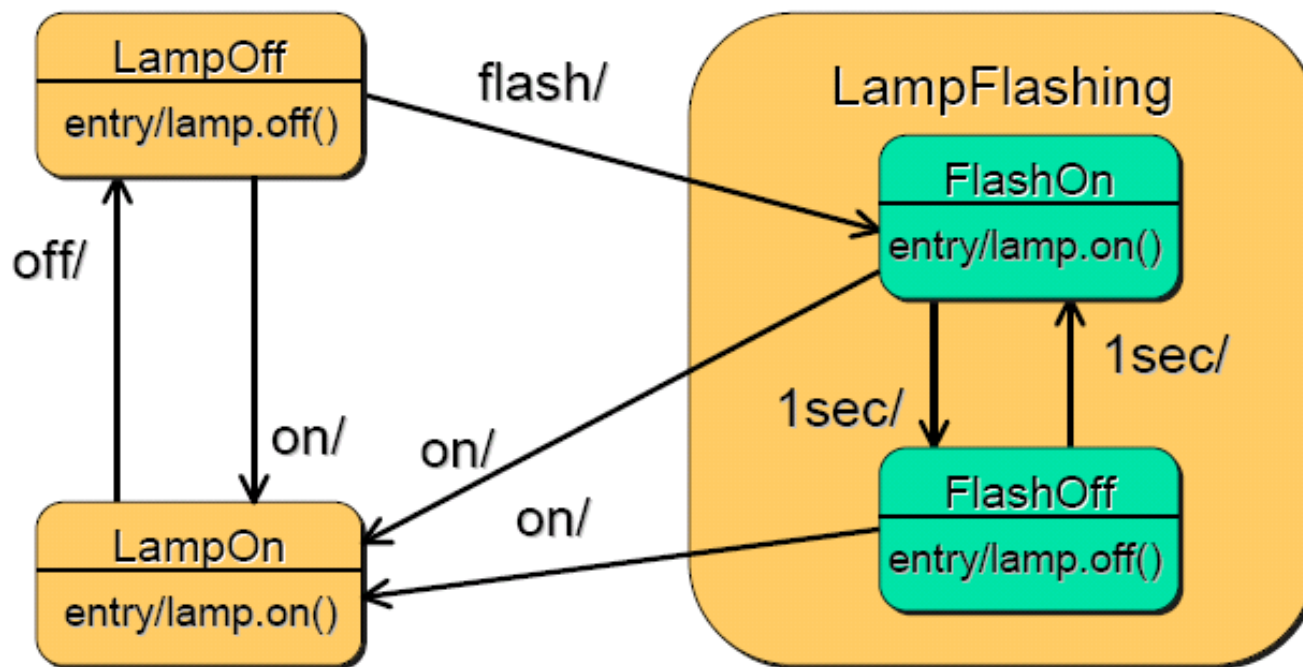
Podmíněné přechody



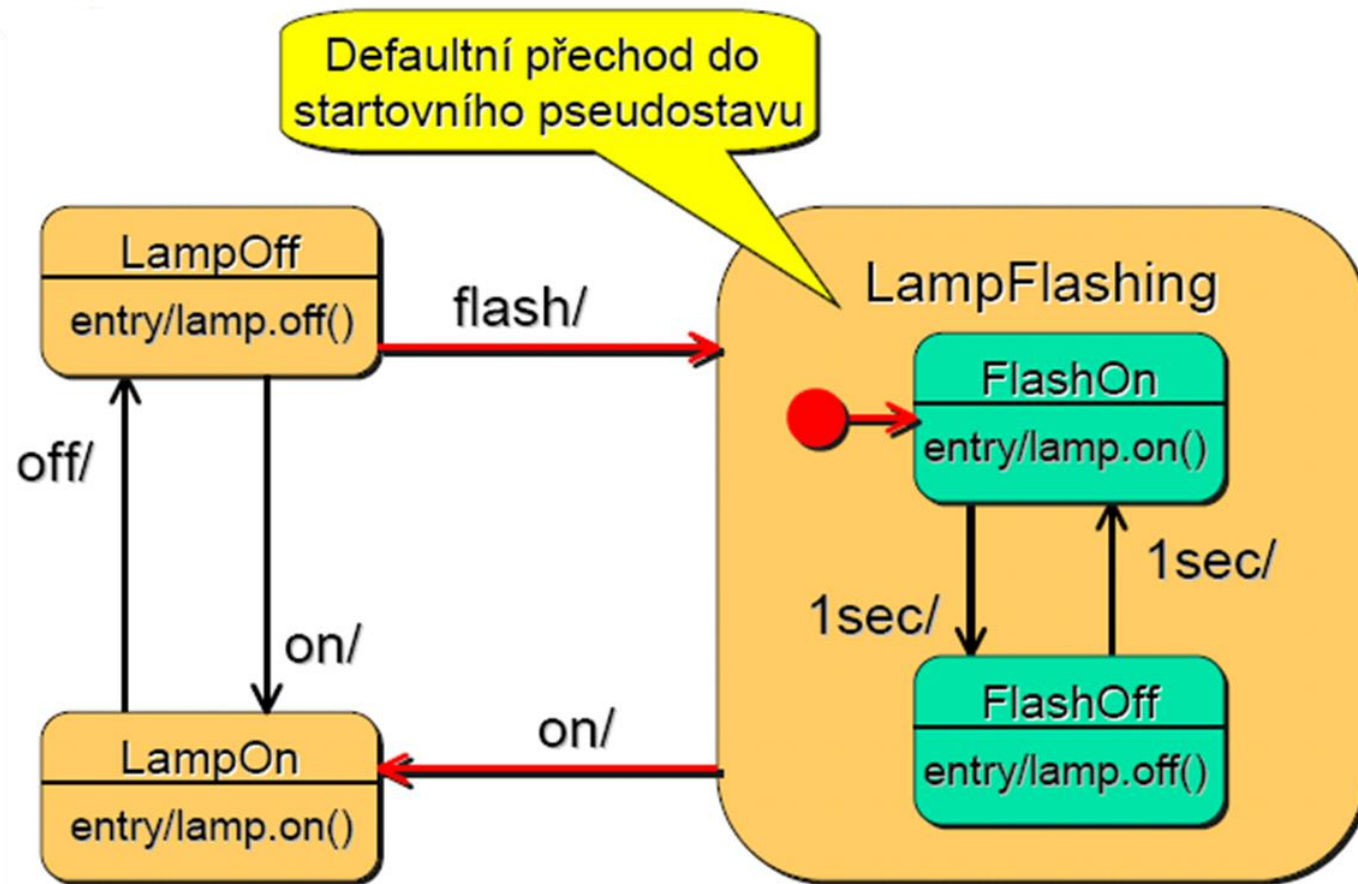
Podmíněné větvení (zkratka)



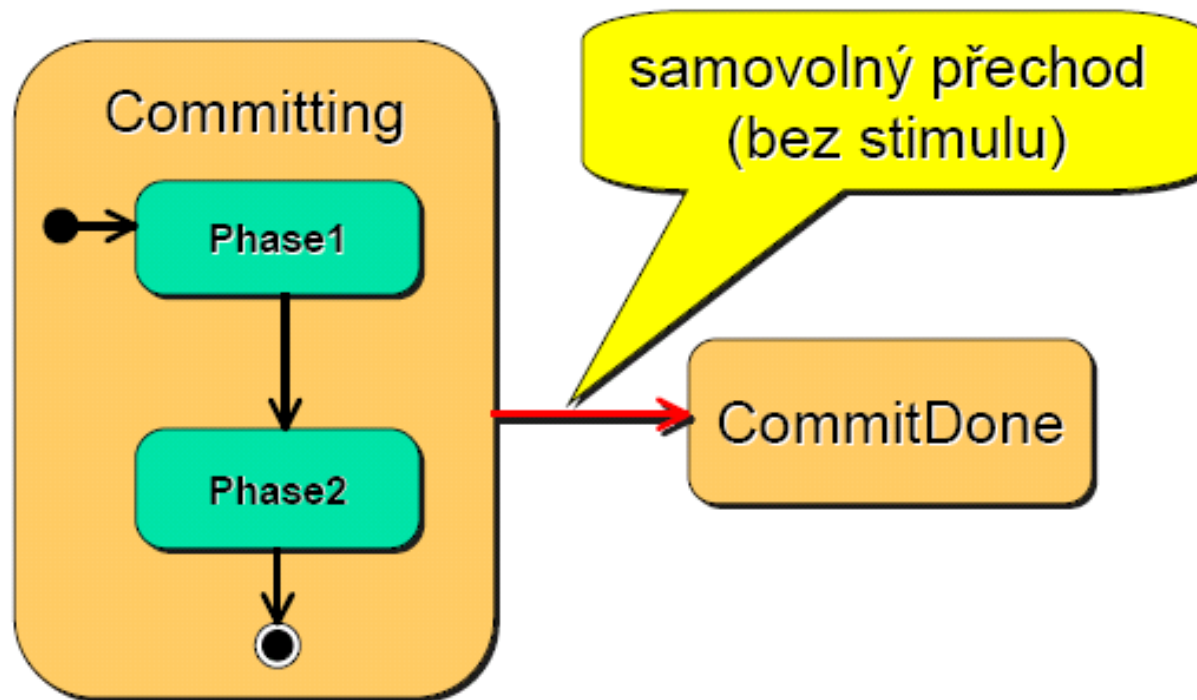
Hierarchické automaty



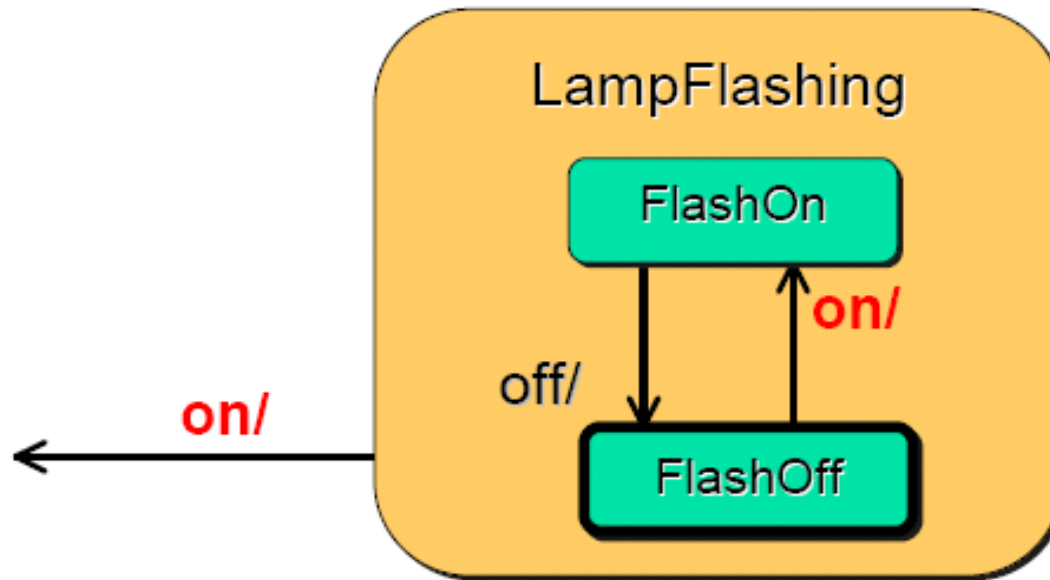
Skupinové přechody



Samovolný přechod

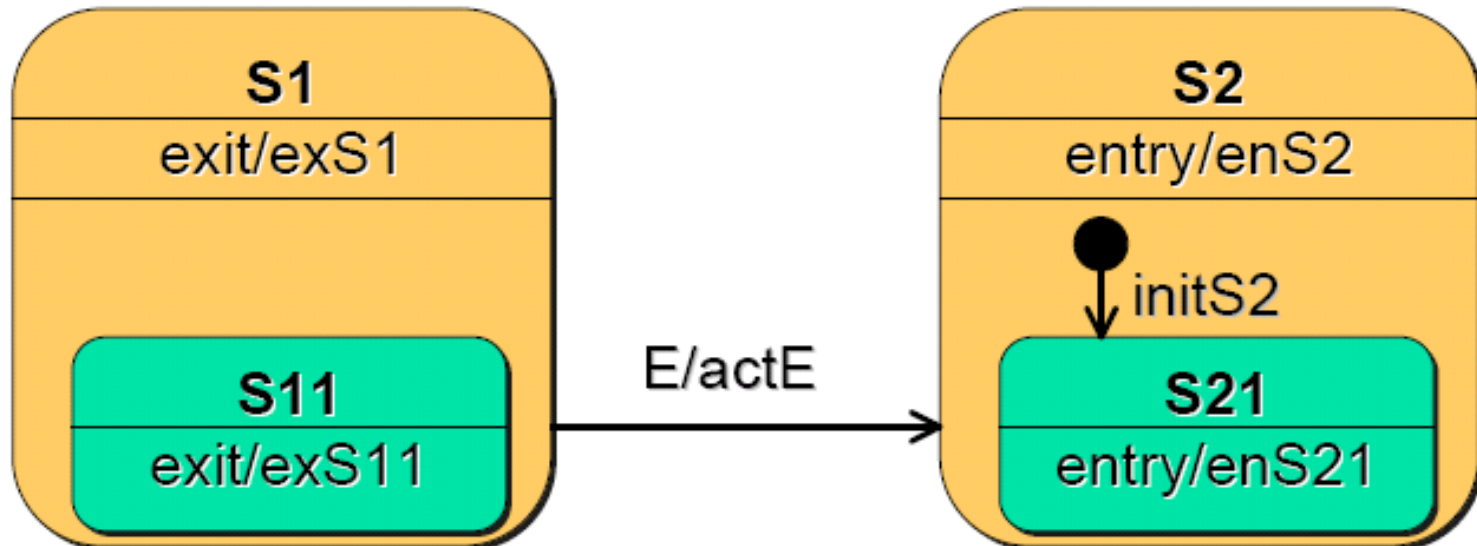


Spouštěcí pravidla



více přechodů může mít stejný signál - vnitřní mají přednost

Pořadí provádění akcí

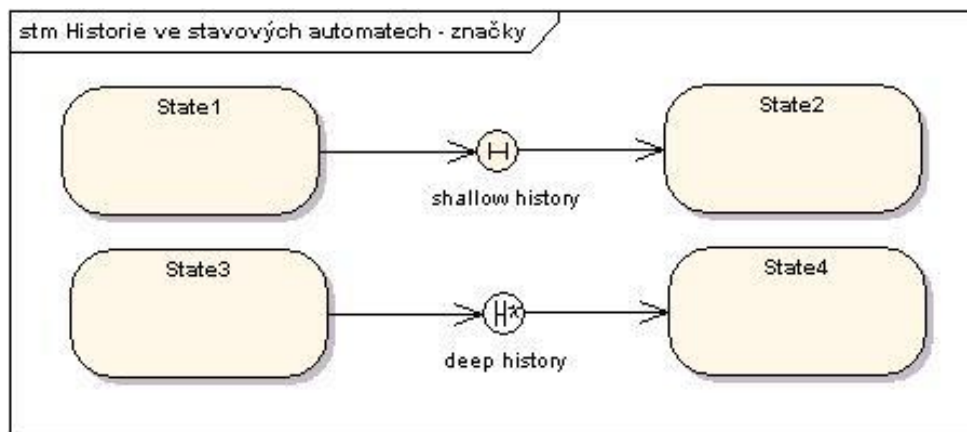


Posloupnost provedení akcí:

exS11 $\Rightarrow$ exS1 $\Rightarrow$ actE $\Rightarrow$ enS2 $\Rightarrow$ initS2 $\Rightarrow$ enS21

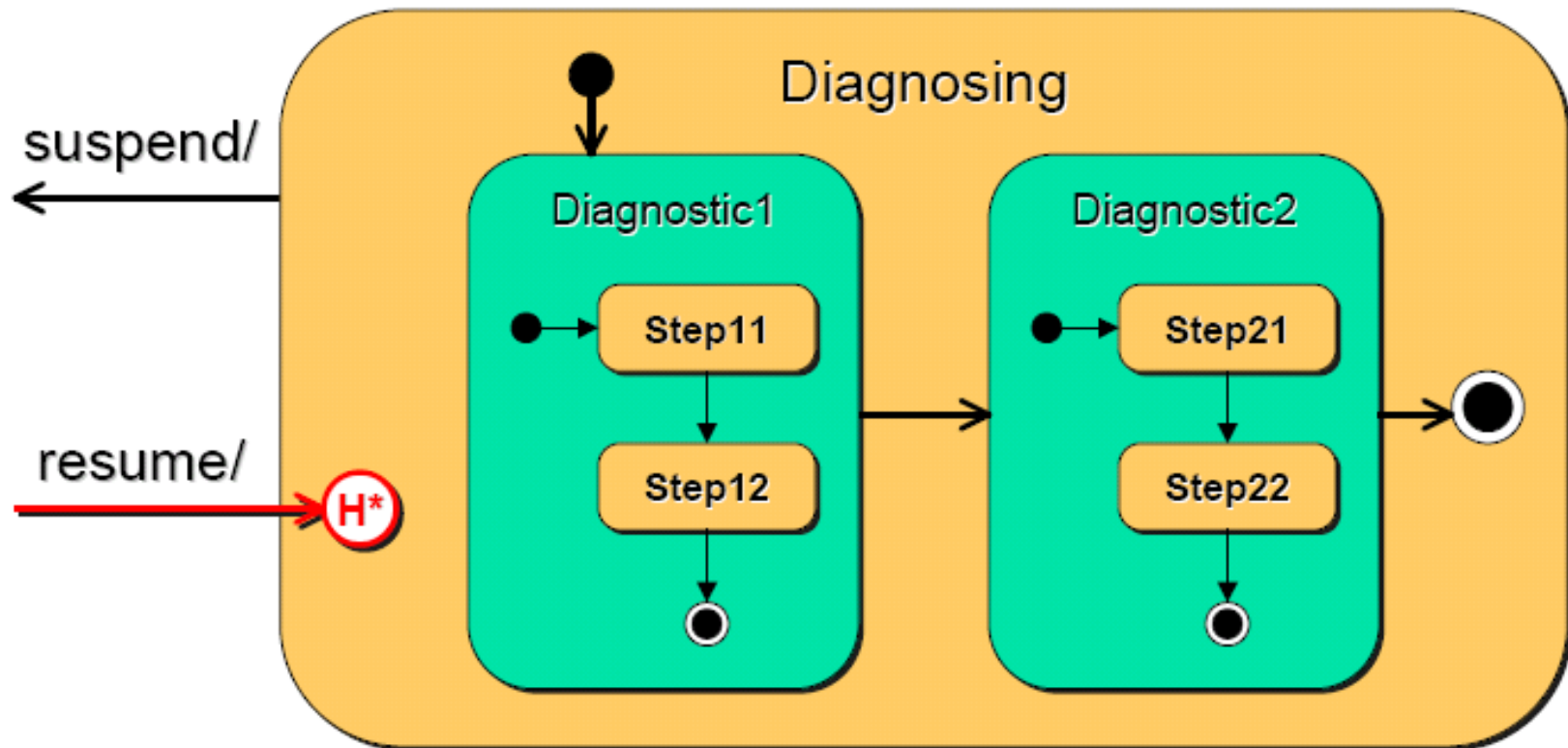
Historie ve stavových diagramech

- Stavové diagramy nyní mohou obsahovat pseudostav "History" (nebo taky "Shallow history") a "Deep history", které by měly vyznačovat možnosti systému se vrátet zpět do historie

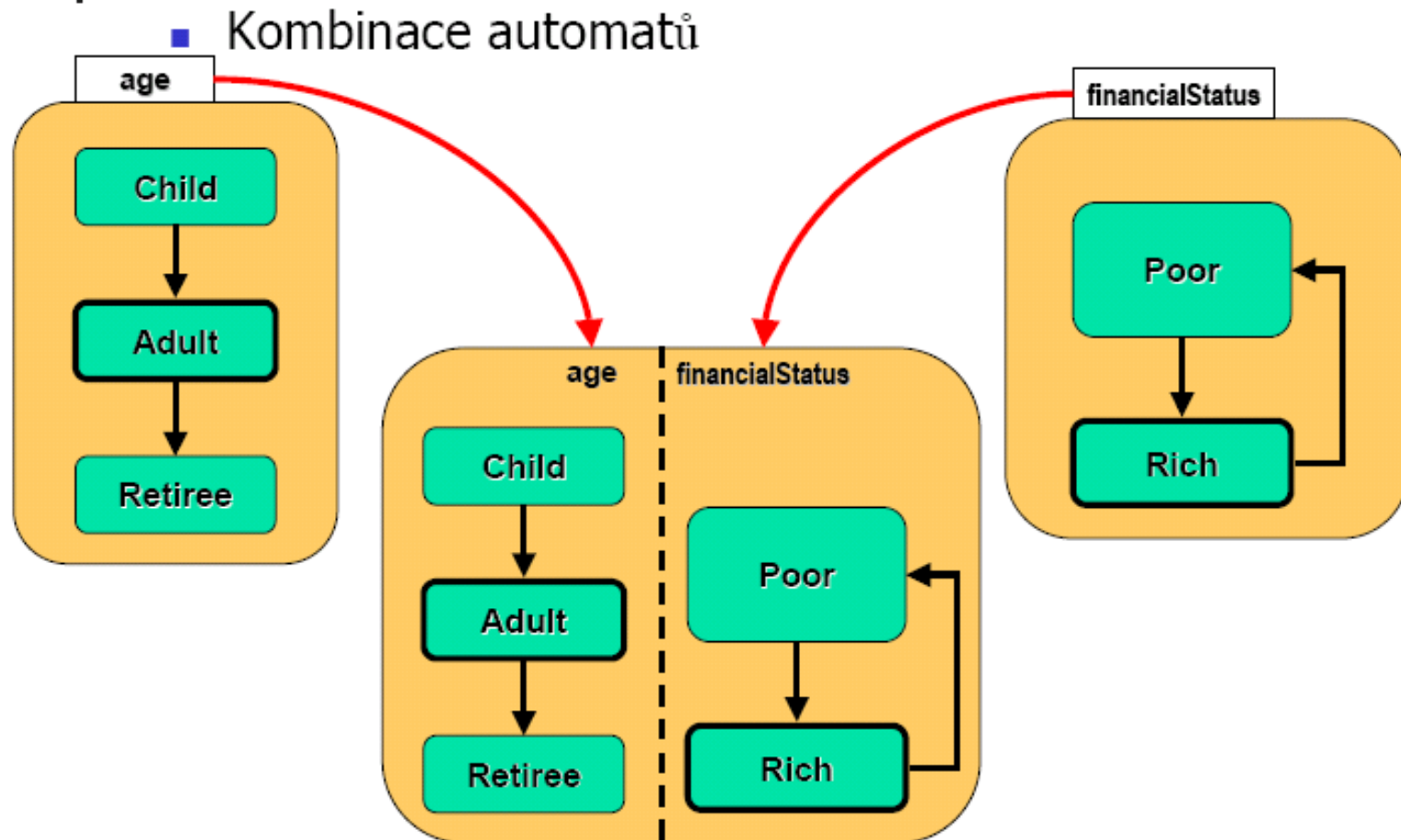


Historie

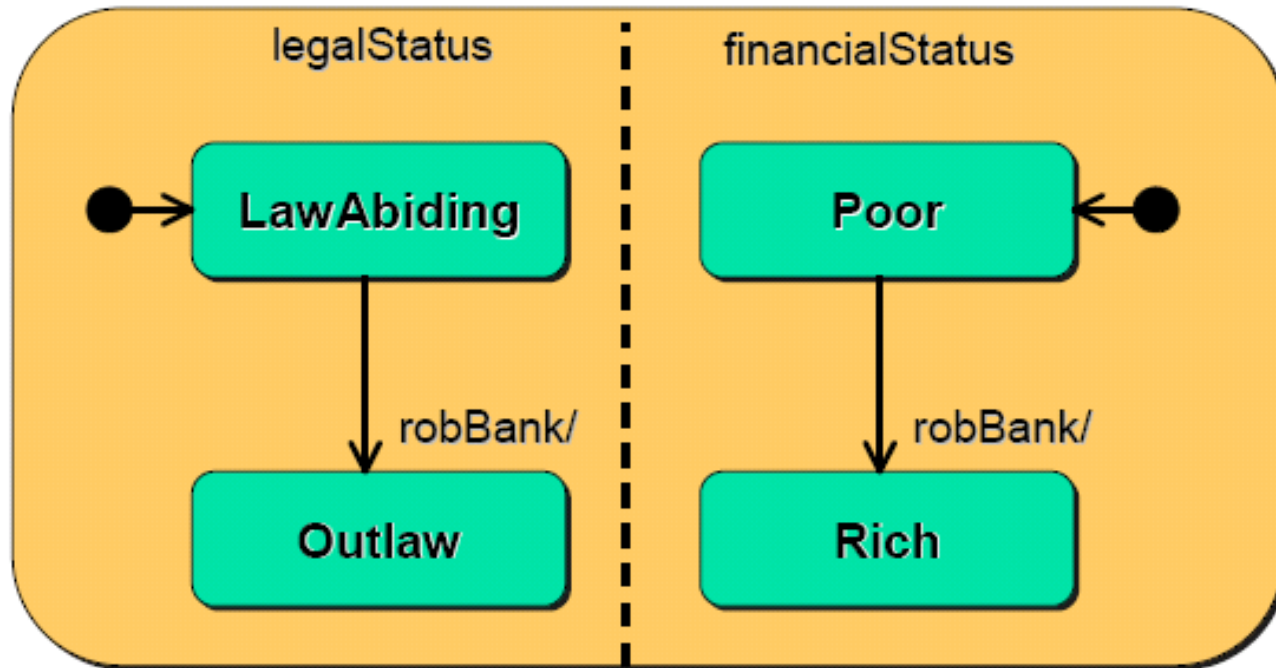
- hluboká a mělká historie



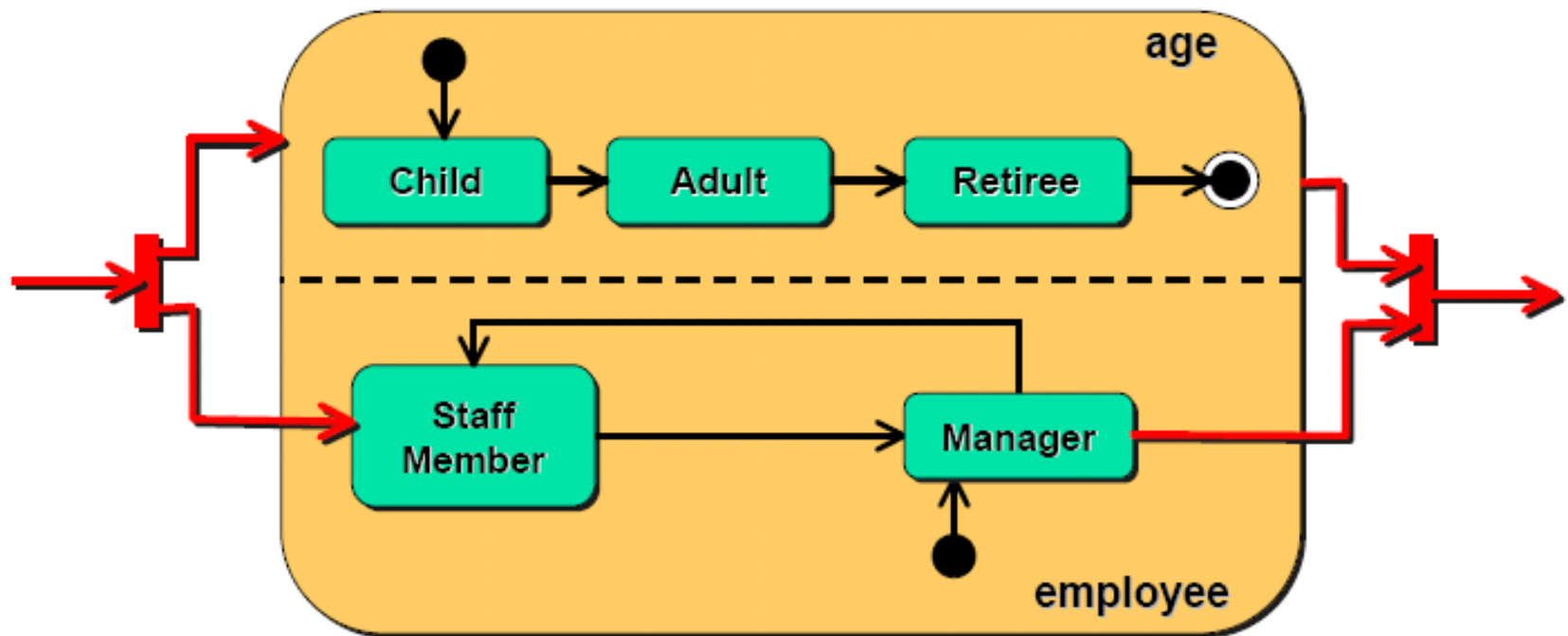
Ortogonalní regiony



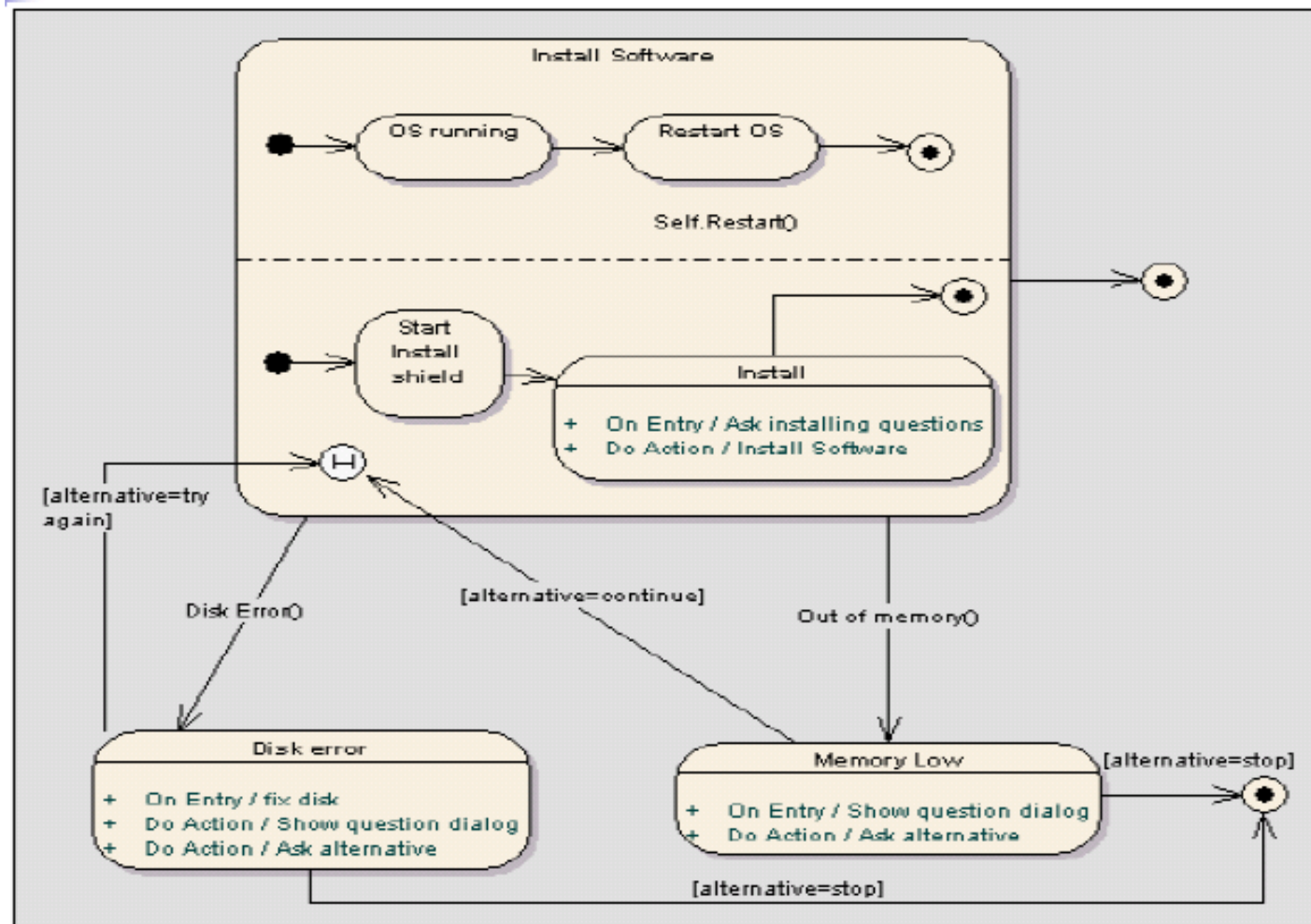
Automaty v regionech běží paralelně



Čeká se na doběhnutí obou automatů



Příklad



Doplňky ke stavovým diagramům

- Přejechod může být ohodnocen:

událost(parametry)[podmínka]/akce^zpráva

- Každý stav může obsahovat popis akcí pro události vstup, výstup a opakované provádění:

entry/akce

exit/akce

do/akce

- Stavové diagramy mohou být hierarchické
- Mohou obsahovat synchronizační značky

Diagramy komponent

- Vyjadřují (fyzickou) strukturu **komponent** systému
- Popisují typy komponent - instance komponent jsou vyjádřeny v diagramu nasazení
- Komponenty mohou být vnořeny do jiných komponent
- Při vyjadřování **vztahu** mezi komponentami lze používat „interface“

Definice komponenty

- UML 1.5 – Modulární, nasaditelná a nahraditelná část systému, která zapouzdřuje implementaci a zveřejňuje množinu rozhraní. Specializace klasifikátoru.
- UML 2.0 – Modulární část systému, která zapouzdřuje svůj obsah a jejíž projev je nahraditelný v daném prostředí. Specializace strukturované třídy (Part, Konektor, Port).

Příklad diagramu komponent

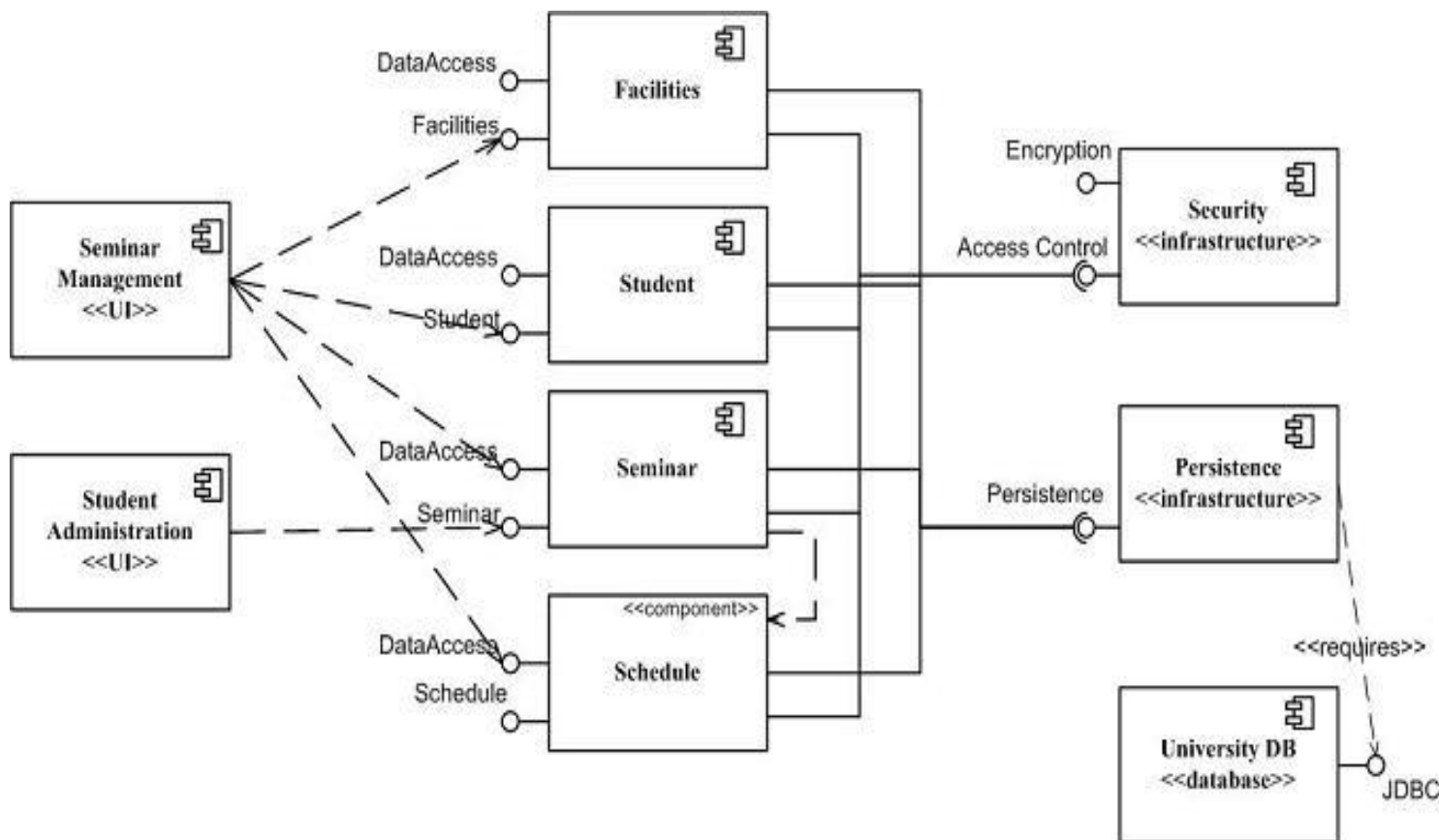
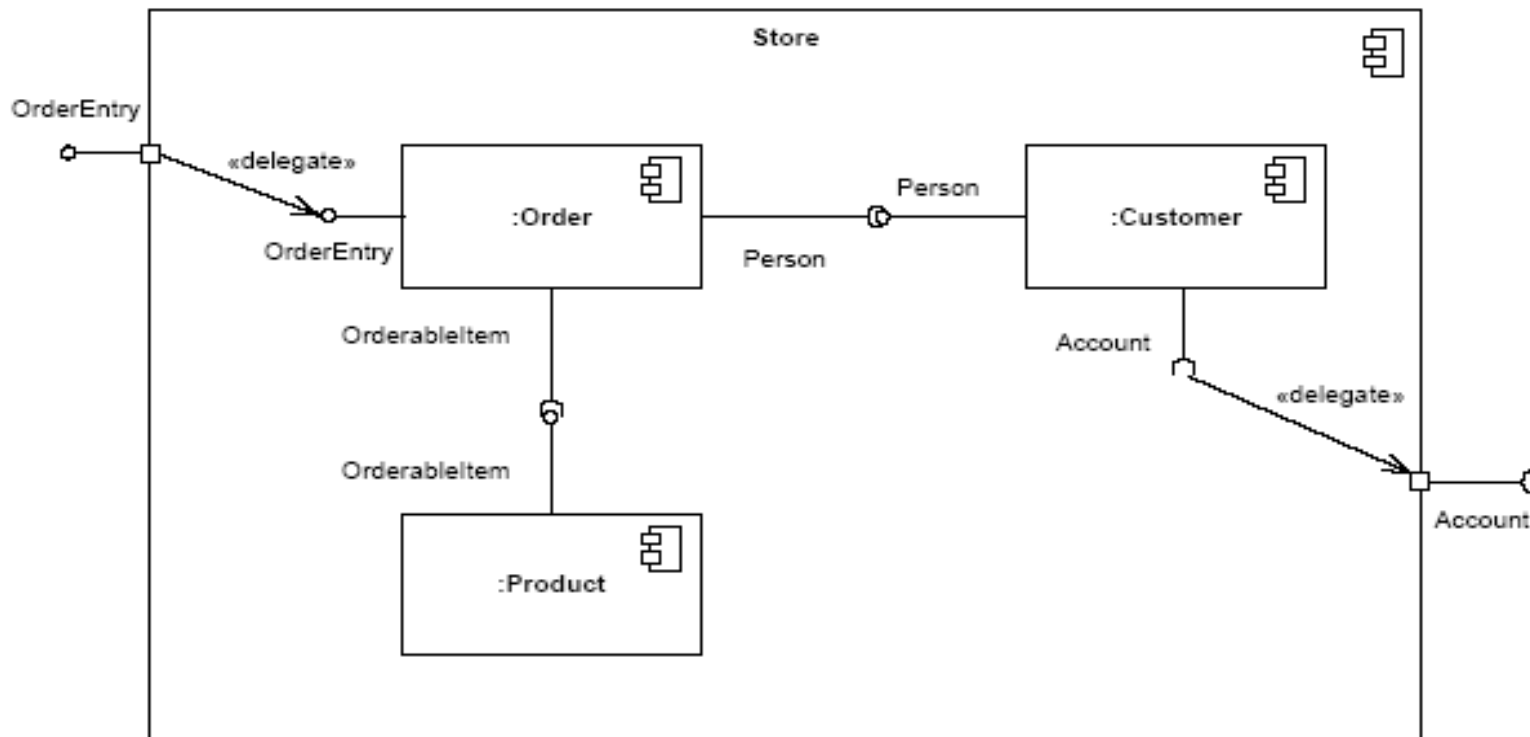


Diagram komponent



Datový model

- notace
- konceptuální model tříd nebo ER-model (diagramy + textový popis)
- další integritní omezení, která nejsou zachycena v diagramech
- datový slovník

Datově orientovaná analýza

- Seznam událostí, kontext, datový slovník
- Identifikace dat, která s událostmi souvisí (identifikace základních objektů)
- Identifikace vztahů mezi objekty
- Scénáře jednání (původce, událost, akce, participant, výstupy - reakce)
- Modelování životních cyklů objektů
- Popis akcí (minispecifikace základních akcí)

Datově orientovaná analýza

- Vychází z představy, že základem IS jsou data. Služby IS slouží pro pořízení a exploraci dat.
- Doporučuje proto nejprve analyzovat požadavky a definovat konceptuální datový model řešeného systému.
- Konceptuální datový model musí postihovat data přicházející přes hranici systému jako vstupní data související s událostmi, dále data, která se v systému ukládají a nakonec rovněž data, která systém produkuje na výstupu.
- Teprve později doplníme model o další části.

Postup datově orient. analýzy

1. Seznam událostí, kontext, datový slovník
2. Identifikace dat, která s událostí souvisí (základních objektů)
3. Identifikace vztahů mezi objekty
4. Scénáře jednání (původce, událost, akce, participant, výstupy - reakce)
5. Modelování životních cyklů objektů
6. Popis akcí (minispecifikace základních akcí)

Jak hledat data?

Doporučení č.1:

- Analyzujeme odborný článek, vybereme všechna podstatná jména.
- Roztřídíme je do skupin:
 - kandidáti na typy objektů (entity),
 - kandidáti na vlastnosti objektů (atributy),
 - ostatní (kandidáti na aktéry, smetí).

Příklad: Odborný článek pro „Výtah“

Systém “Výtah” slouží pro **logické řízení** obsluhy výtahu s jednou či více **šachtami** (předpokládají se 4 šachty a 40 **úrovní**). Systém zajišťuje efektivní plánování sběru a odvozu **pasažérů** mezi obsluhovanými **patry** podle **požadavků** (**požadavek na přivolání** výtahu pro jízdu směrem nahoru nebo dolů, **požadavek na dopravení** do určitého patra). **Směr jízdy** se nemění, dokud výtah nesplní **objednávky** v daném směru (výtah neví o pasažérech – neexistuje **indikace prázdnosti klece**). Přeplněný výtah nereaguje na **výzvy** (existuje **indikace přetížení**). Pro každou šachtu existuje samostatný **motor** ovládaný **signály** (**povely UP, DOWN a STOP**). Povel STOP způsobí zastavení výtahu v nejbližším patře v daném směru a otevření **dveří výtahu** (dveře se dají otevřít až v patře). Uvnitř **klece** je **panel s tlačítky pater**, **indikace aktuální polohy** a **tlačítko STOP**. Tlačítko STOP zabrání zavření dveří (jde mimo systém). Rovněž otevírání a zavírání dveří jde mimo systém (kvůli bezpečnosti). **Příkazy pro systém** jsou akceptovány až po zavření dveří. **Operátor výtahu** má k dispozici **tlačítko ON/OFF**, kterým zadává **požadavek na zastavení pohybu** výtahů.

Zpracovaný článek

systém “Výtah”

logické řízení

šachta

úroveň

pasažér

patro

požadavek

požadavek na přivolání výtahu pro
jízdu směrem nahoru

požadavek na přivolání výtahu pro
jízdu směrem dolů

požadavek na dopravení do patra

směr jízdy

objednávka

indikace prázdnoti klece

výzva

indikace přetížení

motor

signál

povel UP

povel DOWN

povel STOP

dveře výtahu

klec

panel s tlačítky pater

indikace aktuální polohy

tlačítko STOP

příkaz pro systém

operátor výtahu

tlačítko ON/OFF

požadavek na zastavení pohybu

Kandidáti na aktéry

pasažér

indikace přetížení

motor

indikace aktuální polohy (patra)

tlačítko STOP

operátor výtahu

tlačítko ON/OFF

Kandidáti na typy dat

šachta (atribut klece)

úroveň alias patro

požadavek alias objednávka alias
příkaz pro systém alias výzva

požadavek na přivolání výtahu
pro jízdu směrem nahoru

požadavek na přivolání výtahu
pro jízdu směrem dolů

požadavek na dopravení do patra

směr jízdy (atribut)

indikace prázdnoty klece
(neexistuje)

indikace přetížení

signál alias povel (pro motor)

povel UP

povel DOWN

povel STOP

klec

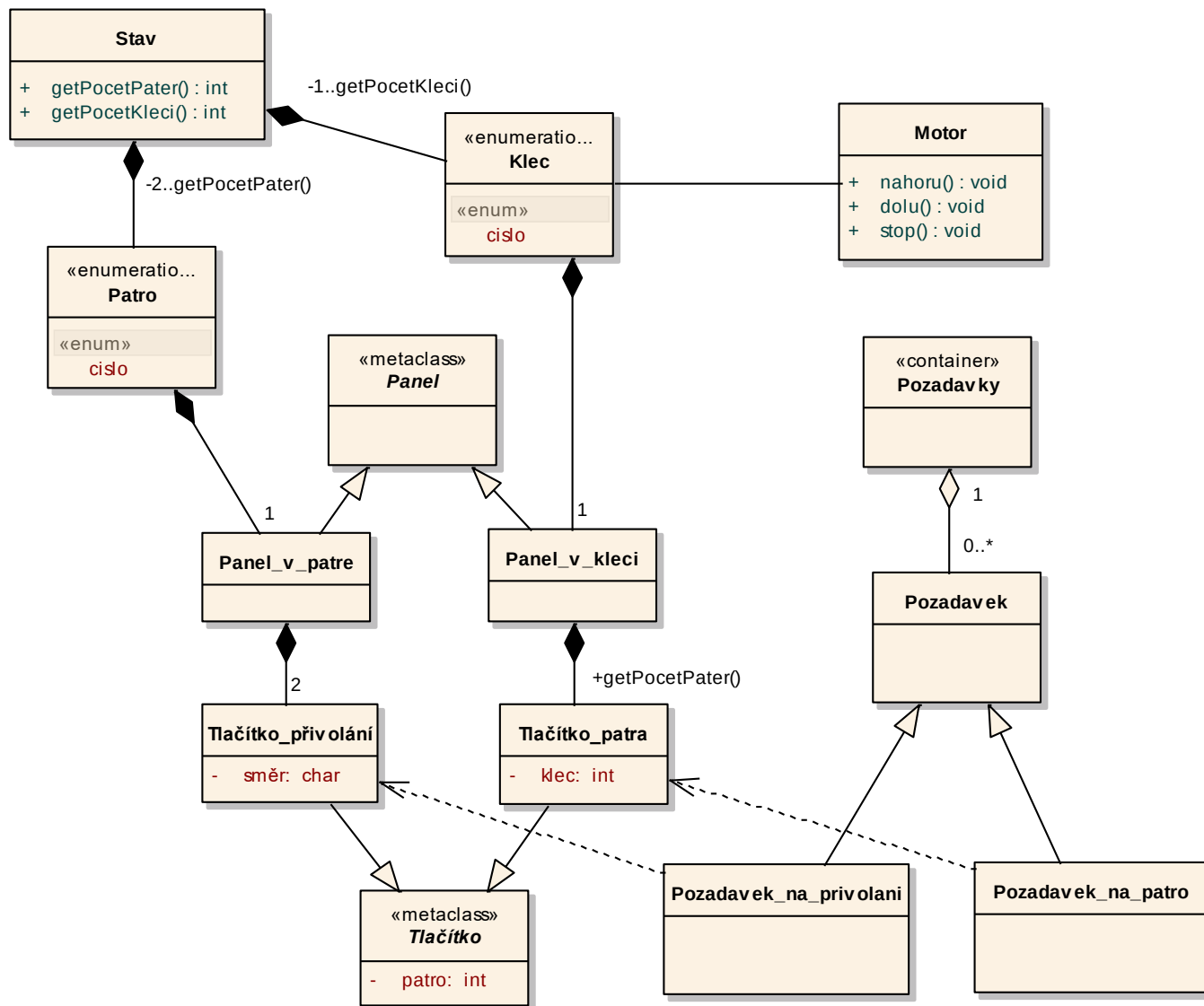
panel s tlačítky pater

indikace aktuální polohy

tlačítko STOP (jde mimo systém)

tlačítko ON/OFF alias požadavek
na zastavení pohybu

class Vytah

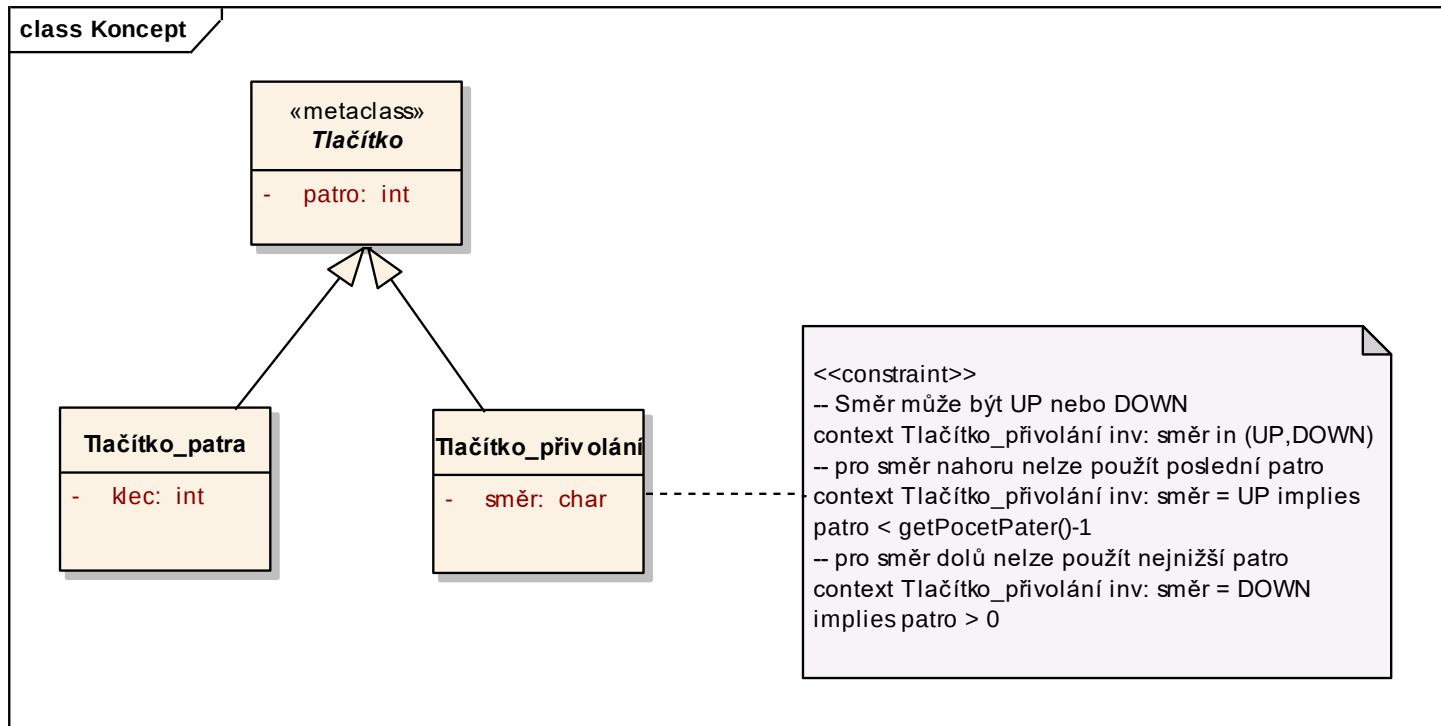


Něco diagramem vyjádřit nelze

V příkladu systému Výtah je to např.:

- Tlačítko pro přivolání pro jízdu směrem nahoru na panelu v posledním patře, tj. když patro má hodnotu `getPocetPater()` neexistuje.
- Tlačítko pro přivolání pro jízdu směrem dolů na panelu v prvním patře neexistuje.

Připojení omezení k prvkům



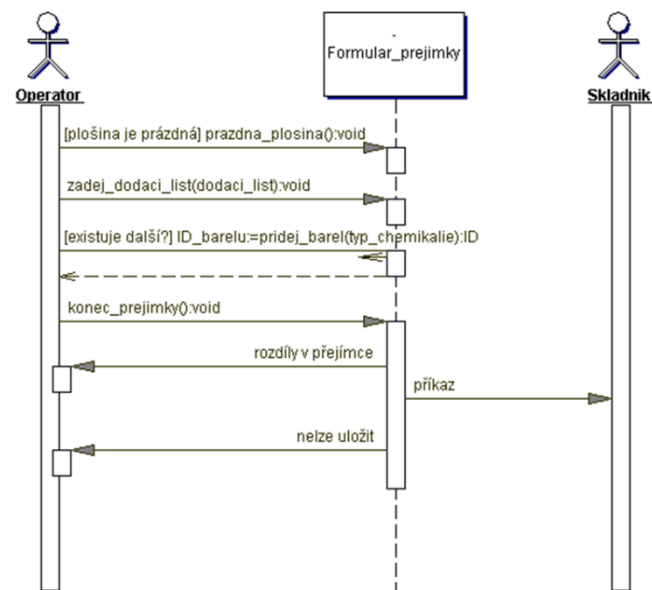
Jak hledat data?

Doporučení č.2:

- Analyzujeme seznam událostí, rozpoznáváme data, která s událostmi souvisí.
- Roztřídíme je do skupin:
 - kandidáti na typy objektů (entity),
 - kandidáti na vlastnosti objektů (atributy).

Příklad: Události pro ECO sklad

- Operátor zahájil přejímku
- Operátor zahájil dodávku
- Manažer se ptá na stav skladu
- Manažer se ptá na bezpečnost skladu



Kandidáti na typy dat

dodací list

barel

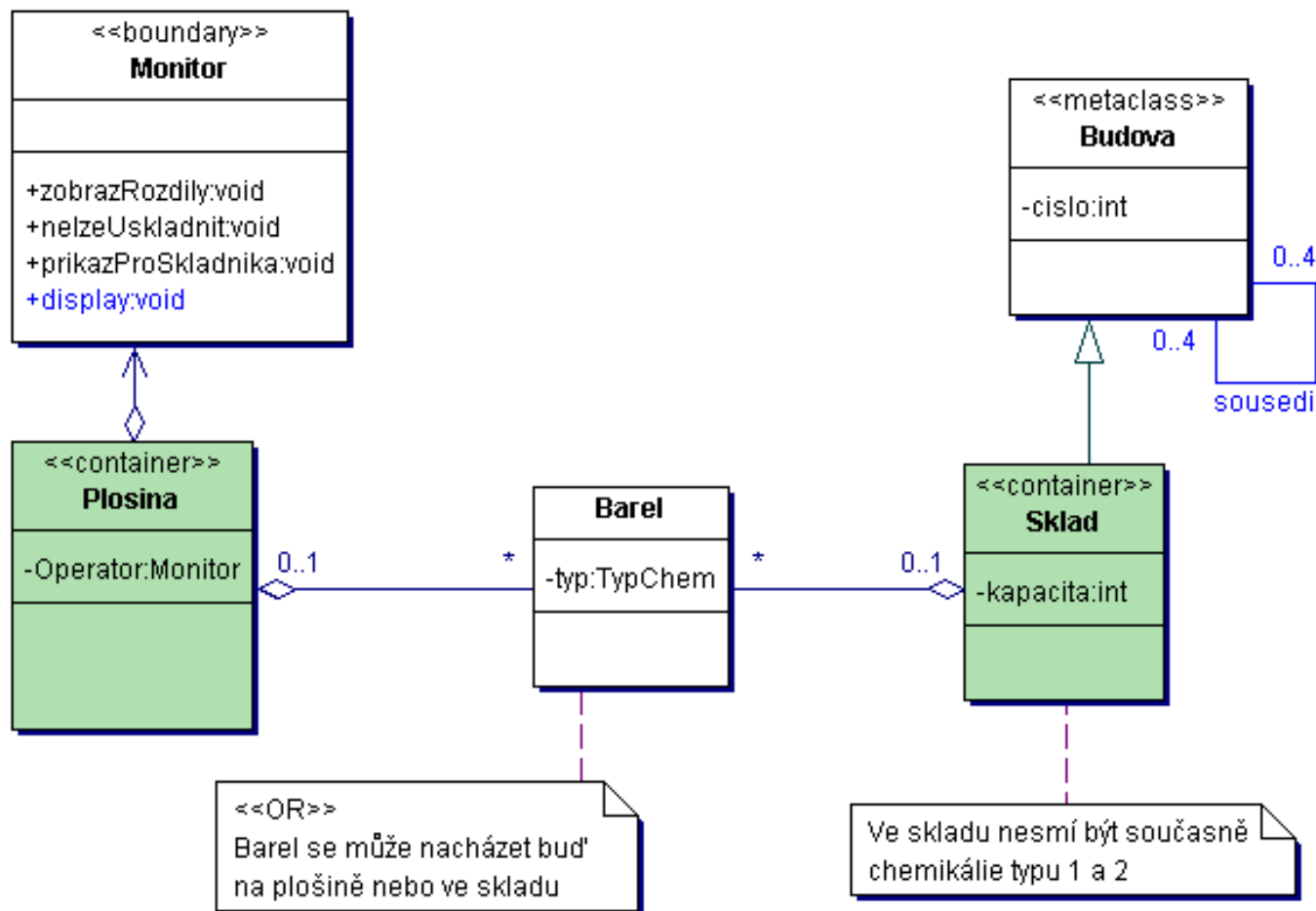
typ chemikálie

rozdíly v přejímce

nelze uložit

příkaz pro skladníka

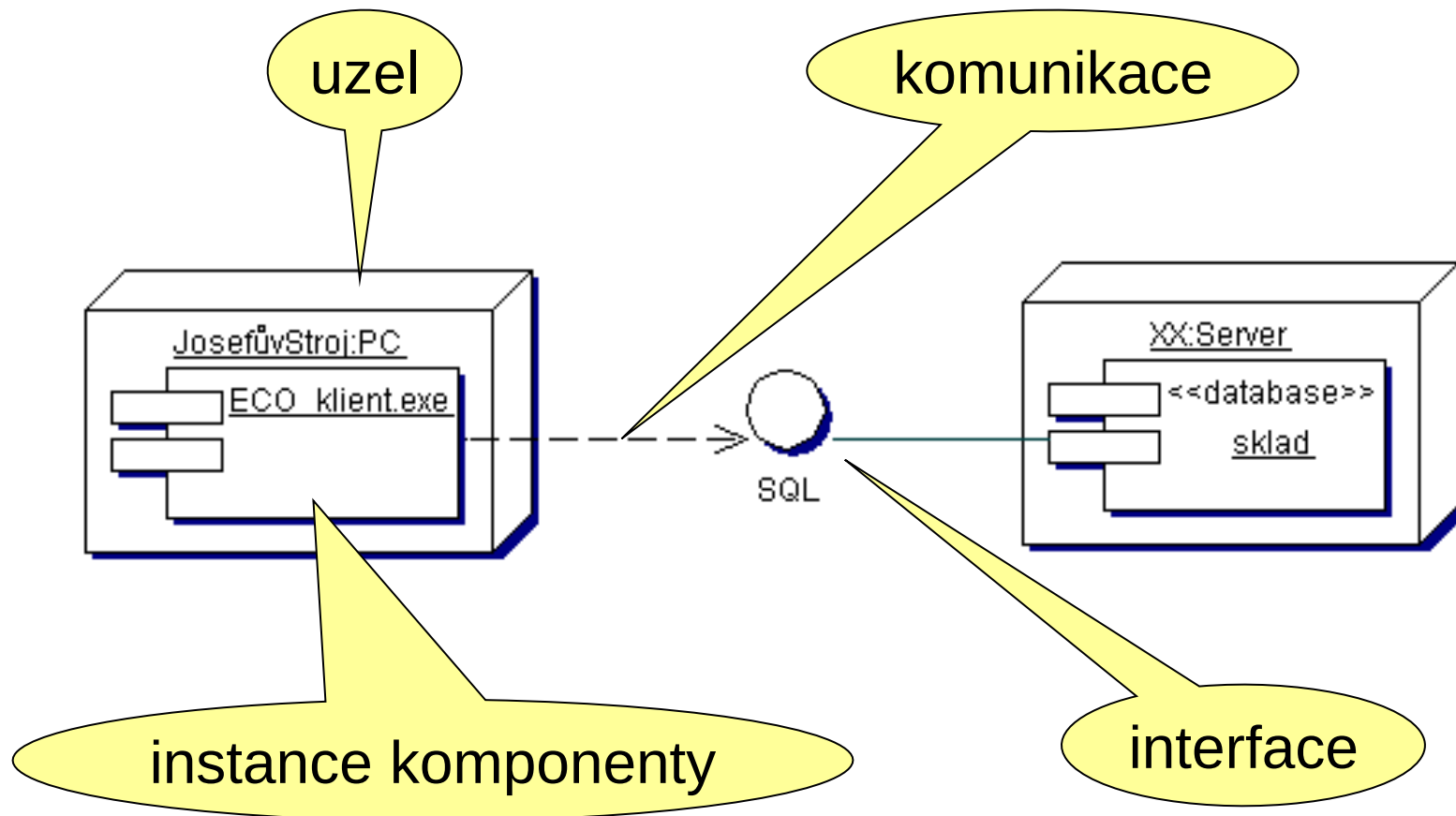
Datový model pro ECO-sklad (1.)



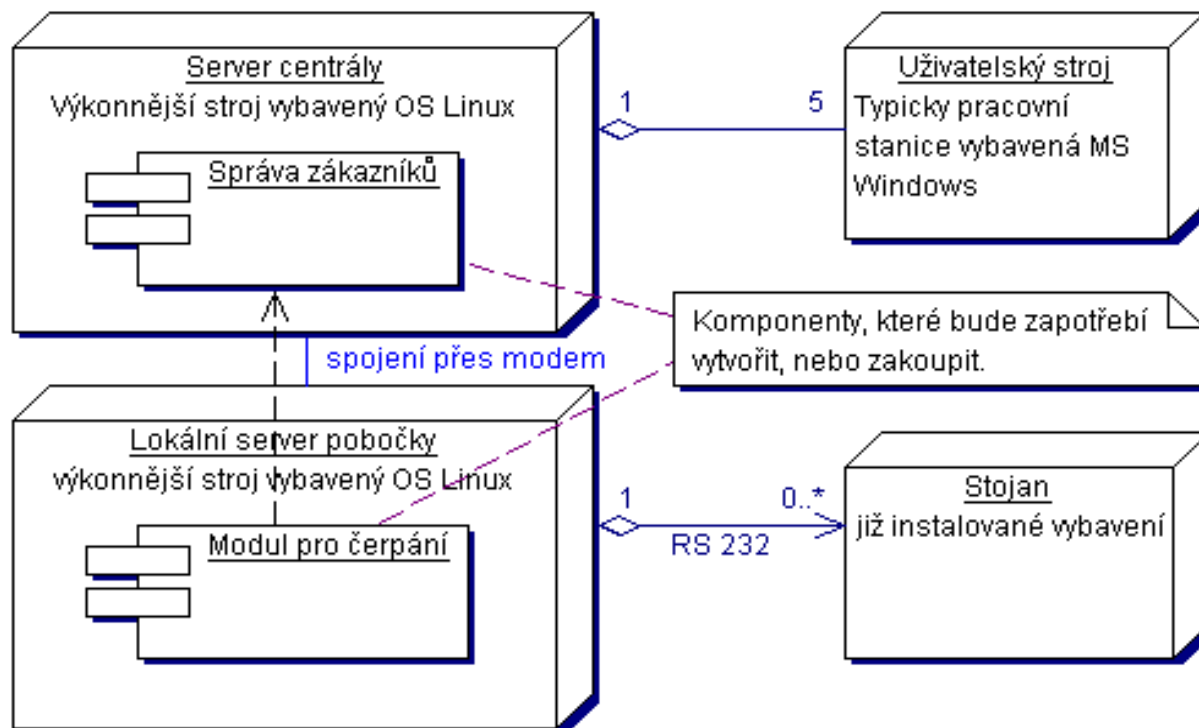
Diagramy nasazení

- Popisují fyzické rozmístění elementů systému na **uzly** výpočetního systému
- Uzly a elementy jsou značeny obdobně jako objekty a třídy (může být uveden pouze typ, nebo konkrétní instance a typ - podtržena)
- Popisují nutné **vazby** mezi uzly (případně též použitý protokol - „interface“)
- Obsahují pouze **komponenty** potřebné pro běh aplikace - komponenty potřebné pro překlad a sestavení jsou v diagramu komponent

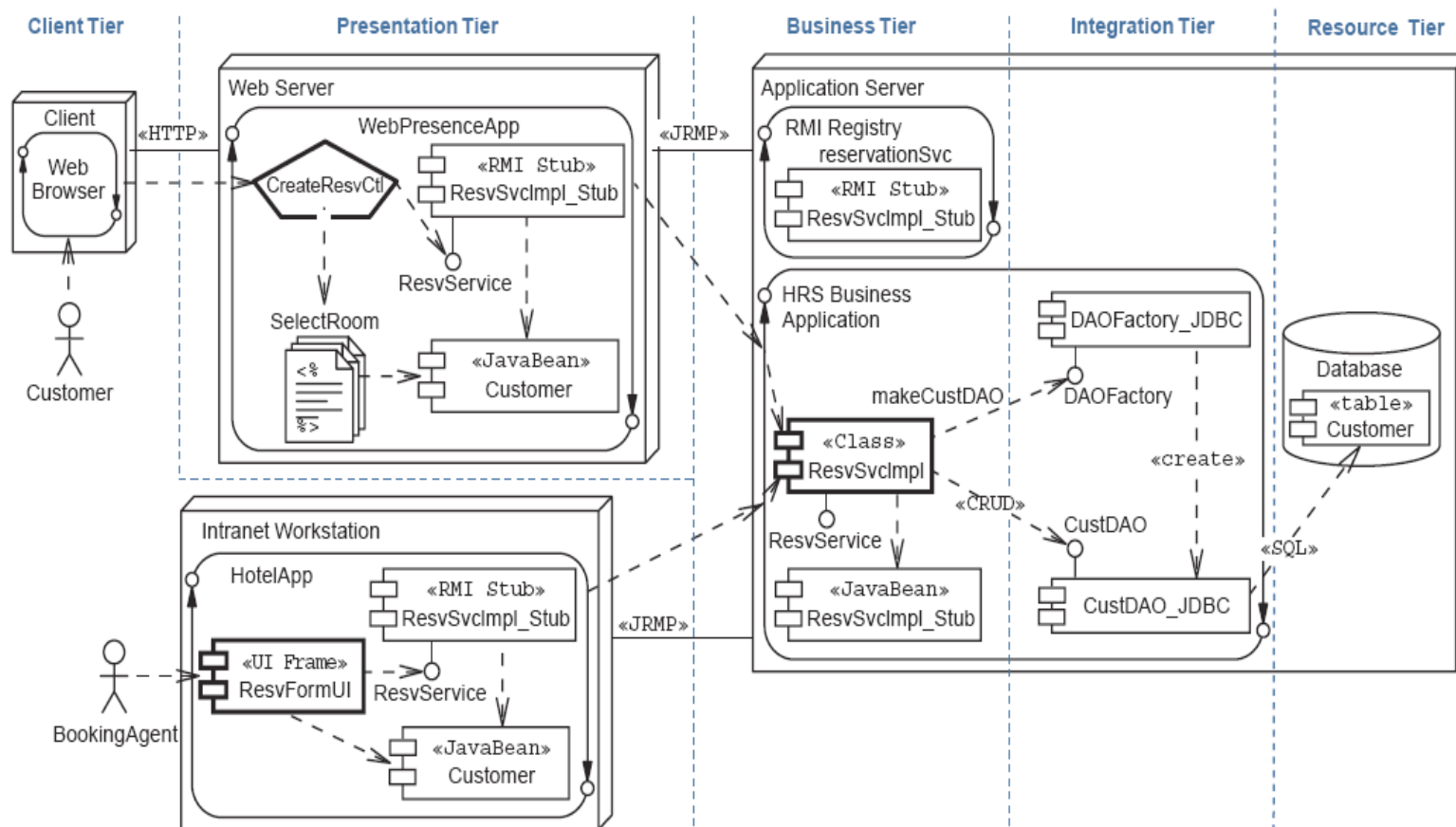
Příklad diagramu nasazení



Příklad diagramu nasazení



Příklad diagramu nasazení



The End